Studio Nx

P1-14 내 프로젝트 화면 JS — 함수 설명 합본 (JS 29~42)

2026-10-08

목차

1. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/29_myproject_formatRelativeTime.md
2. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/30_myproject_loadProjects.md
3. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/31_myproject_renderProjects.md
4. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/32_myproject_renderPager.md
5. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/33_myproject_initSearch.md
6. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/34_myproject_initRowMenus.md
7. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/35_studio_canOpenStudio.md
8. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/36_studio_showDesktopOnly.md
9. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/37_studio_openStudio.md
10. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/38_studio_initDesktopOnlyLinks.md
11. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/39_myproject_initNewProjectForm.md
12. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/40_myproject_initEditProjectForm.md
13. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/41_myproject_initDeleteProject.md
14. ai-notes/javascript/P1-14 - 내 프로젝트 화면 JS/42_myproject_loadInitialList.md

JS 29 — `formatRelativeTime()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면 — 목록의 "마지막 수정" 칸 글자 · PRD 01 FR-123
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	28 `initContactForm()` · 30 `loadProjects()`

1. 하는 일

서버의 수정 시각(`updated_at`)을 사람이 읽기 쉬운 글자로 바꾼다. 목록 오른쪽 "언제" 칸에 들어가는 글자다 (HTML 예시와 같은 모양).

수정한 지	글자
1분 안 (또는 미래 — 컴퓨터 시계가 조금 느릴 때)	`Just now`
1분 · 12분	`1 minute ago` · `12 minutes ago`
1시간 · 11시간 (24시간 안)	`1 hour ago` · `11 hours ago`
달력으로 어제	`Yesterday`
2 ~ 7일 전	`2 days ago` … `7 days ago`
8일 이상 · 올해	`Sep 30`
다른 해	`Dec 31, 2025`

2. 언제 쓰나

호출하는 곳	화면
`renderProjects` (31, 다음 함수들)	목록 한 줄의 `[data-field="when"]` 칸 — 예: `Site Song … 12 minutes ago`

서버 시각의 모양: 서버는 UTC 시각을 **시간대 표시 없이** 보낸다 — `"2026-10-08T02:10:11.123000"`. 자바스크립트 `new Date()` 는 시간대 표시가 없는 날짜+시각 글자를 **이 컴퓨터의 시간대**로 읽으므로, 그대로 쓰면 서울에서는 9시간 어긋난다. 그래서 끝에 `Z`(UTC 라는 표시)를 붙여 읽는다. userInfo 의 가입일(`formatDate`, P1-10 22)과 같은 방법이다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout } from '../core/auth.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import 를 쓰지 않는다 — 날짜 계산은 브라우저 기본 기능(`Date`, `toLocaleDateString`)만으로 한다. (import 정리는 다음 함수들에서)

함수

```
export function formatRelativeTime(isoTime, now = new Date()) {
  const hasZone = /[zZ]$|[+-]\d\d:\d\d$/.test(isoTime);
  const time = new Date(hasZone ? isoTime : isoTime + 'Z');
  const minutes = Math.floor((now - time) / 60000);

  if (minutes < 1) return 'Just now';
  if (minutes < 60) return minutes === 1 ? '1 minute ago' : minutes + ' minutes ago';
  const hours = Math.floor(minutes / 60);
  if (hours < 24) return hours === 1 ? '1 hour ago' : hours + ' hours ago';

  // 달력으로 며칠 전인지 — 두 날의 자정끼리 비교 (시각은 버림)
  const today = new Date(now.getFullYear(), now.getMonth(), now.getDate());
  const day = new Date(time.getFullYear(), time.getMonth(), time.getDate());
  const days = Math.round((today - day) / 86400000);   // round: 서머타임으로 하루가 23·25시간인 날 대비
  if (days <= 1) return 'Yesterday';
  if (days <= 7) return days + ' days ago';

  const options = { month: 'short', day: 'numeric' };
  if (time.getFullYear() !== now.getFullYear()) options.year = 'numeric';
  return time.toLocaleDateString('en-US', options);
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
if (minutes < 60) return `${minutes} minute${minutes === 1 ? '' : 's'} ago`;
…
return time.toLocaleDateString('en-US', {
  month: 'short', day: 'numeric',
  ...(time.getFullYear() !== now.getFullYear() && { year: 'numeric' }),
});
```

최신 문법	실제 코드	뜻
`` `${minutes} minute…` `` (템플릿 문자열)	`minutes + ' minutes ago'`	글자 안에 값 넣기
`...(조건 && { year: 'numeric' })`	`if (조건) options.year = 'numeric'`	조건이 맞을 때만 칸 추가

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	`Date` · `Math` · `toLocaleDateString` 은 브라우저에 원래 있음

식	뜻	예
`now = new Date()` (매개변수 기본값)	안 넘기면 지금 시각. 시험할 때만 시각을 넘김	`f(iso)` = 지금 기준, `f(iso, 정오)` = 정오 기준
`/[zZ]$\|[+-]\d\d:\d\d$/.test(isoTime)`	끝이 `Z` 또는 `+09:00` 처럼 시간대로 끝나는지	`"...02:48:00"` → false, `"...02:48:00Z"` → true
`now - time`	두 `Date` 를 빼면 **밀리초** 차이	12분 = 720000

식	뜻	예
`/ 60000` · `Math.floor`	밀리초 → 분, 소수점 버림	11분 59.9초 → 11
`new Date(년, 월, 일)`	그날 자정 (이 컴퓨터 시간대)	시각을 버리고 날짜만 비교하려고
`/ 86400000` · `Math.round`	밀리초 → 일 (하루 = 86 400 000ms), 반올림	서머타임 날 3.04 → 3
`toLocaleDateString('en-US', { month: 'short', day: 'numeric' })`	영어 날짜 글자	`Sep 30`, year 를 더하면 `Dec 31, 2025`

4. 흐름

1. 시간대 표시가 없으면 `z` 를 붙여 UTC 로 읽는다.
2. 지금과의 차이(분)를 구한다 → 1분 안 · 60분 안 · 24시간 안이면 그 글자.
3. 24시간이 넘으면 **달력**으로 며칠 전인지 센다 → 어제 · 7일 안 · 그 밖은 날짜.

5. 왜 이렇게

선택	이유
`z` 를 붙여 읽음	서버 시각은 UTC 인데 표시가 없다. 붙이지 않으면 서울에서 "9 hours ago" 가 "Just now" 로 나오는 식으로 어긋남
이미 시간대가 있으면 그대로	나중에 서버가 `z` 를 붙여 보내도 `zz` 가 되어 깨지지 않게
24시간 안은 **시간**, 넘으면 **달력**	밤 12시 30분에 본 어젯밤 11시 수정은 "Yesterday" 보다 "1 hour ago" 가 정확. 하루가 넘으면 "어제 · 3일 전" 이 사람이 생각하는 방식
미래 시각 → `Just now`	컴퓨터 시계가 서버보다 조금 느리면 방금 만든 프로젝트가 "-1 minutes ago" 로 나올 수 있음
`Math.round` (일)	서머타임이 바뀌는 날은 하루가 23 · 25시간 → 나누면 2.96 · 3.04 가 나옴. 미국 · 유럽 사용자 대비
7일 넘으면 날짜, 다른 해면 연도까지	템플릿 · HTML 예시와 같음. "작년 12월 31일" 을 "Dec 31" 로만 쓰면 올해로 착각
`now` 매개변수	시험에서 "지금" 을 고정하려고. 화면에서는 넘기지 않음
`'en-US'` 고정	시켠서 밖 페이지는 영어만 (2026-10-07 결정)

6. 시험 (실제 myProject.html · 헤드리스 Chrome · 20개)

페이지 안에서 `import('/static/js/pages/myproject.js')` 로 함수를 불렀다. 시간대는 Chrome 설정으로 고정했다.

서울 (지금 = 2026-10-08 12:00 서울 = 03:00 UTC)

서버 시각 (UTC)	결과	확인한 것
`…03:00:00` · `…02:59:30` · `…03:05:00`	`Just now` ×3	같은 시각 · 30초 · 미래 5분
`…02:59:00`	`1 minute ago`	단수
`…02:48:00` · `…02:47:59.500000` · `…02:48:00Z` · `…11:48:00+09:00`	`12 minutes ago` ×4	서버 형식(소수 6자리) · `Z` · `+09:00` 모두 같게
`…02:48:00.123000`	`11 minutes ago`	11분 59.877초 → 버림

서버 시각 (UTC)	결과	확인한 것
`…02:00:00` · `10-07T15:30:00`	`1 hour ago` · `11 hours ago`	서울로는 오늘 00:30
`10-07T02:00:00` · `10-06T15:00:00`	`Yesterday` ×2	25시간 · 서울 어제 자정
`10-06T14:59:00`	`2 days ago`	서울 그저께 23:59 (1분 차이로 이틀)
`10-01T03:00:00` · `09-30T03:00:00`	`7 days ago` · `Sep 30`	7일까지 "days ago", 8일부터 날짜
`2025-12-31T03:00:00`	`Dec 31, 2025`	다른 해

뉴욕 (서머타임이 끝나는 11월 1일 사이)

경우	결과
10월 31일 정오(EDT) → 11월 3일 정오(EST) — 사이에 25시간짜리 하루	`3 days ago`
23.5시간 전 (날짜로는 어제)	`23 hours ago`
`now` 생략 — 방금 시각	`Just now`

처음 돌렸을 때 1개가 기대와 달랐는데, 함수가 아니라 **시험의 기대값**이 틀렸다 (`02:48:00.123` 은 11분 59.877초 전이라 `11 minutes ago` 가 맞음). 기대값을 고치고, 12분이 되는 서버 형식 시각(`02:47:59.500000`)을 따로 넣었다.

JS 30 — `loadProjects()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면 — 목록 받기 · PRD 01 FR-123
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	29 `formatRelativeTime()` · 31 `renderProjects()`

1. 하는 일

검색어 · 정렬 · 페이지로 **내 프로젝트 목록을 서버에서 받고**, 그 조건을 기억하고 **주소창에도 남긴다**. 화면에 그리는 일은 31 `renderProjects` 가 한다.

부른 값	서버 요청	주소창
`loadProjects()`	`/projects?query=&sort=updated&page=1&size=10`	`myProject.html`
`{ query: ' gam ', sort: 'title' }`	`query=gam&sort=title…`	`myProject.html?query=gam&sort=title`
`{ page: 2 }`	`page=2`	`myProject.html?page=2`
`{ sort: 'random', page: 'abc' }` (손으로 친 주소)	`sort=updated&page=1` 로 고쳐서	`myProject.html`
`{ query: 'a&b%' }`	`query=a%26b%25` (안전하게 바꿔 보냄)	`?query=a%26b%25`

실패	옮겨 가는 곳
401 (로그인 풀림)	`home.html#login?next=/myProject.html` — 로그인 뒤 돌아옴
500 · 502 등 500대	`errors/500.html` (503 · 504 는 같은 번호)
서버에 닿지 못함 (네트워크)	`errors/503.html`
그 밖의 4xx (예: 422)	`errors/400.html`

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (38)	처음 열 때 — 주소의 `?query=&sort=&page=` 로
검색 · 정렬 (33)	검색 칸에 치거나 정렬을 바꿀 때 (page 1부터)
페이지 넘김 (32)	Prev · 1 2 3 · Next
새 프로젝트 · 삭제 뒤 (35 · 37)	목록 다시 받기 — `current` 의 조건 그대로

`history.replaceState`: 페이지를 새로 열지 않고 **주소창 글자만** 바꾸는 브라우저 기능. 새로고침해도 같은 검색 결과가 나오고, 주소를 복사해 보내도 같은 화면이 열린다. `pushState` 와 달리 뒤로 가기 기록을 쌓지 않아, 글자 하나 칠 때마다 뒤로 가기가 늘어나지 않는다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

`loginUrl` 과 `api` 줄을 이번에 추가했다. 이 함수는 `api`(서버 요청, P1-02) · `loginUrl`(로그인 주소 만들기, P1-02)를 쓴다.

상수 (이번에 추가)

```
/** 정렬 값 — #project-sort 의 option value · 서버 ProjectSort 와 같음 */
const SORTS = ['updated', 'title', 'created'];

/** 검색어 최대 길이 — 서버 query max_length 와 같음 */
const QUERY_MAX = 100;

/** 같은 번호의 오류 화면이 있는 상태 코드 (frontend/errors/) — userinfo.js 와 같음 */
const ERROR_PAGES = [400, 403, 404, 429, 500, 503, 504];

/** 지금 보고 있는 목록 조건 — 페이지 넘김 · 삭제 뒤 다시 받을 때 씀 (loadProjects 가 갱신) */
export const current = { query: '', sort: 'updated', page: 1 };
```

함수

```
export async function loadProjects({ query = '', sort = 'updated', page = 1 } = {}) {
  // 1. 값 정리 — 주소창에 손으로 친 값(?page=abc, ?sort=random)도 서버가 422 를 내지 않게
  query = String(query).trim().slice(0, QUERY_MAX);
  if (SORTS.indexOf(sort) < 0) sort = 'updated';
  page = parseInt(page, 10);
  if (!(page >= 1)) page = 1;   // NaN · 0 · 음수 → 1

  // 2. 서버에 요청
  const params = new URLSearchParams({ query: query, sort: sort, page: String(page), size: String(PAGE_S
IZE) });
  const ticket = ++loadTicket;
  let data;
  try {
    data = await api.get('/projects?' + params.toString());
  } catch (err) {
    if (ticket !== loadTicket) return null;   // 더 새 요청이 있으면 이 실패는 무시
    if (err.status === 401) {
      location.replace(loginUrl(location.pathname + location.search + location.hash));
    } else {
      let errorPage = err.status >= 500 ? 500 : 400;
      if (ERROR_PAGES.indexOf(err.status) >= 0) errorPage = err.status;
      else if (err.status === 0) errorPage = 503;
      location.replace('errors/' + errorPage + '.html');
```

```
    }
    return null;
  }

  if (ticket !== loadTicket) return null;   // 그사이 더 새 요청을 보냄 → 이 응답은 버림 (33)

  // 3. 지금 조건 기억 + 주소에 남김 (기본값은 빼서 주소를 짧게) — 새로고침 · 뒤로 가기에서 유지
  current.query = query;
  current.sort = sort;
  current.page = page;
  const shown = new URLSearchParams();
  if (query) shown.set('query', query);
  if (sort !== 'updated') shown.set('sort', sort);
  if (page !== 1) shown.set('page', String(page));
  const search = shown.toString();
  history.replaceState(null, '', location.pathname + (search ? '?' + search : '') + location.hash);

  return data;
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const params = new URLSearchParams({ query, sort, page, size: PAGE_SIZE });
…
const shown = new URLSearchParams({
  ...(query && { query }),
  ...(sort !== 'updated' && { sort }),
  ...(page !== 1 && { page }),
});
history.replaceState(null, '', `${location.pathname}${search ? `?${search}` : ''}${location.hash}`);
if (!SORTS.includes(sort)) sort = 'updated';
```

최신 문법	실제 코드	뜻
`{ query, sort }` (줄임 속성)	`{ query: query, sort: sort }`	이름과 값이 같은 칸
`...(조건 && { query })`	`if (조건) shown.set('query', query)`	조건이 맞을 때만 칸 넣기
`SORTS.includes(sort)`	`SORTS.indexOf(sort) < 0`	목록에 있는지
템플릿 문자열 `` `${a}?${b}` ``	`a + '?' + b`	글자 잇기

문법 풀이

import 문법	뜻
`import { api } from '../core/api.js'`	`api.get(주소)` — 토큰을 붙여 보내고, 만료되면 한 번 재발급 후 다시 보냄. 실패하면 `ApiError`(status · code · message)를 던짐 (P1-02)
`import { loginUrl } from '../core/auth.js'`	`loginUrl('/myProject.html')` → `'home.html#login?next=%2FmyProject.html'` (P1-02 · P1-07)

식	뜻	예
`{ query = '', sort = 'updated', page = 1 } = {}`	객체 하나를 받아 칸을 꺼내고, 없으면 기본값. 아무것도 안 넘겨도 됨	`loadProjects({ page: 2 })` → query `''` · sort `'updated'`
`String(query).trim().slice(0, QUERY_MAX)`	글자로 바꿔 앞뒤 공백 제거, 100자까지	`' gam '` → `'gam'`

식	뜻	예
`parseInt(page, 10)`	글자 → 정수 (10진수)	`'2'` → 2, `'abc'` → `NaN`
`!(page >= 1)`	1 이상이 **아니면** — `NaN >= 1` 은 false 라서 NaN 도 걸림	`NaN` · 0 · −3 → 1 로
`new URLSearchParams({…}).toString()`	칸들을 `a=1&b=2` 로, `&` · `%` · 공백은 안전한 글자로	`'a&b%'` → `a%26b%25`
`err.status >= 500 ? 500 : 400`	조건 ? 맞을 때 : 아닐 때	502 → 500, 422 → 400
`location.replace(주소)`	그 주소로 옮겨 감. 지금 페이지를 기록에서 **바꿔치기** → 뒤로 가기로 오류 전 페이지에 다시 오지 않음	
`export const current = {…}`	다른 함수(32 · 35 · 37)가 읽는 "지금 조건". `const` 라도 안의 칸은 바꿀 수 있음	`current.page = 2`

4. 흐름

1. 값 정리 — 검색어 공백 제거 · 100자, 정렬은 3가지 중 하나, 페이지는 1 이상 정수.
2. `GET /projects?query=&sort=&page=&size=10`.
3. 실패 → 401 은 로그인으로, 그 밖은 오류 화면으로 옮기고 `null`.
4. 성공 → `current` 에 조건 기억 → 기본값이 아닌 것만 주소에 남김 (`#…` 는 그대로) → 응답을 돌려줌.

5. 왜 이렇게

선택	이유
값을 먼저 정리	주소는 사용자가 손으로 고칠 수 있다 (`?page=abc`). 서버에 보내 422 를 받는 것보다 미리 기본값으로 — 사용자에게는 그냥 첫 페이지가 보임
오류는 `loadMe`(22)와 같은 규칙	템플릿은 "500 이상 → 500" 만 적었지만, 같은 모양의 페이지 데이터 API 라 401 · 503 · 네트워크도 같은 방식으로 (2026-10-06 결정 "본 데이터 API 오류 → 오류 화면")
그 밖의 4xx → 400 화면	값을 정리해서 일어나지 않아야 하지만, 생기면 빈 화면보다 "잘못된 요청" 화면이 낫다
실패하면 `null` 을 돌려줌	부르는 쪽(38 등)이 `if (!data) return;` 한 줄로 멈출 수 있게 — 오류 화면으로 옮겨 가는 중에 그리기를 하지 않음
주소에는 **기본값을 뺀** 것만	`myProject.html?query=&sort=updated&page=1` 보다 `myProject.html` 이 깔끔. 읽을 때(38)는 없으면 기본값
`replaceState` (`pushState` 아님)	검색할 때마다 뒤로 가기 기록이 쌓이지 않게
주소 갱신은 **성공한 뒤**	실패해서 오류 화면으로 가는데 주소만 바뀌는 일이 없게
`current` 를 밖으로 내보냄	페이지 넘김 · 새 프로젝트 · 삭제 뒤 "같은 조건으로 다시 받기" 에 씀
옛 응답 버리기 (`loadTicket`, 33 에서 추가)	빠르게 검색하면 먼저 보낸 요청이 나중에 도착해 새 결과를 덮어쓸 수 있음 → 요청마다 번호를 붙이고 마지막 것만 씀 (33 §5)
`ERROR_PAGES` 를 userinfo.js 와 따로 둠	지금은 두 곳뿐이라 그대로 두었다. 세 번째 페이지(P1-17 myPage)에서도 쓰면 `core/` 로 옮기는 것이 좋다

6. 시험 (실제 myProject.html · 실제 서버 · 헤드리스 Chrome · 14개)

시험 계정 `sitetest.p114@example.com` 으로 가입 · 로그인하고 프로젝트 3개(Beta → alpha → Gamma 순서로 만듦)를 만든 뒤, 페이지 안에서 함수를 불렀다. 오류 시험은 `window.fetch` 를 가짜로 바꿔 서버 응답을 흉내 냈다. 끝난 뒤 계정 · 프로젝트를 지웠다.

#	부른 값	결과 · 주소
01	기본	`Gamma, alpha, Beta` (최근 수정순) · total 3 · 주소 `myProject.html`
02	`query: ' gam ', sort: 'title'`	`Gamma` · current `{query:'gam', sort:'title', page:1}` · `?query=gam&sort=title`
03	`sort: 'random', page: 'abc'`	updated · 1 로 고쳐서 3개 · 주소 깨끗
04	`page: 0` · `page: -3`	둘 다 1
05	`page: 2` (10개씩이라 없는 쪽)	빈 목록 · total 3 · `?page=2`
06	`query: 'a&b%'`	0개 · `?query=a%26b%25` (특수 문자가 주소를 깨지 않음)
07	150자 검색어	100자로 잘라 보냄 → 422 없이 0개
08	주소 `?query=old&page=9#top` 에서 `sort: 'created'`	`?sort=created#top` — 옛 값은 지우고 `#top` 은 남김
09	가짜 500	`errors/500.html`
10	가짜 503	`errors/503.html`
11	가짜 502	`errors/500.html`
12	가짜 422	`errors/400.html`
13	네트워크 끊김 (fetch 실패)	`errors/503.html`
14	로그아웃 뒤	`home.html#login?next=%2FmyProject.html`

JS 31 — `renderProjects()` (+ `buildRow`)

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js` · (CSS) `static/css/pages/myproject.css`
작업	P1-14 내 프로젝트 화면 — 목록 그리기 · PRD 01 FR-123 · FR-125 · FR-130
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	30 `loadProjects()` · 32 `renderPager()`

1. 하는 일

`loadProjects`(30)가 받은 목록을 **화면의 줄로 그린다**. HTML 의 예시 줄 7개를 지우고, `<template id="project-row-template">` 을 프로젝트 수만큼 복사해 채운다.

칸	넣는 것	예
제목	제목 + 안 열었으면 `Not opened` 표	`Lo-fi Sketch [Not opened]`
설명	설명, 없으면 `No description`	
BPM · Sig · Key	`Number(bpm)` · `7/8` · 키 없으면 `–`	`'128.50'` → `128.5`, `'120.00'` → `120`
Members	점 최대 3개 + OWNER 가 아니면 역할	5명 EDITOR → ●●● `Editor`
Edited	`formatRelativeTime(updated_at)` (29) + 휴대폰용 `data-meta`	`12 minutes ago`, meta `128 BPM · A minor ·`
… 메뉴	OWNER 가 아니면 Delete 잠금 + `Owner only`, VIEWER 는 Edit 도 잠금	
링크	제목 · Open → `studioNx.html?projectId=번호`	

목록	화면
1개 이상	개수 `2 projects` (1개면 `1 project`) · 제목 줄 · 목록
0개 · 검색어 없음	`No projects yet. Create your first project`
0개 · 검색어 있음	`No projects match your search.`

개수는 **이 페이지의 줄 수가 아니라 전체**(`total`) — 23개 중 1쪽이면 `23 projects`.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 · 검색 · 페이지 넘김 · 새 프로젝트 · 삭제 뒤 (32 · 33 · 35 · 37 · 38)	항상 `loadProjects` 다음에 — `const data = await loadProjects(…); if (data) renderProjects(data);`

`<template>`: 화면에 보이지 않는 HTML 틀. `template.content` 안의 요소를 `cloneNode(true)` 로 복사하면 같은 모양의 새 줄이 생긴다. 줄 모양은 HTML 이 정하고 JS 는 값만 채운다 (나중에 React 로 옮길 때도 틀을 그대로 컴포넌트로).

3. 코드

import (`myproject.js` 맨 위 — 30 과 같음, 이번에 추가한 것 없음)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import 를 쓰지 않는다 — 같은 파일의 `current`(30) · `formatRelativeTime`(29) 만 쓴다.

함수 (줄 하나 만들기는 `buildRow` 로 나눔 — 파일 밖에서는 안 씀)

```
export function renderProjects(data) {
  const list = document.getElementById('project-list');
  const template = document.getElementById('project-row-template');
  const count = document.getElementById('project-count');
  const label = document.querySelector('.pj-list .pj-row--label');
  const empty = document.getElementById('project-empty');
  const noResult = document.getElementById('project-no-result');
  if (!list || !template) return;

  // 1. 개수 · 빈 목록 안내
  if (count) count.textContent = data.total === 1 ? '1 project' : data.total + ' projects';
  const isEmpty = data.items.length === 0;
  list.hidden = isEmpty;
  if (label) label.hidden = isEmpty;
  if (empty) empty.hidden = !(isEmpty && !current.query);
  if (noResult) noResult.hidden = !(isEmpty && current.query);

  // 2. 줄 그리기 — 예시 줄을 지우고 틀을 복사해 채움
  list.textContent = '';
  for (let i = 0; i < data.items.length; i++) {
    list.appendChild(buildRow(template, data.items[i]));
  }
}

/** 프로젝트 하나 → 목록 한 줄 <li> (renderProjects 가 씀) */
function buildRow(template, item) {
  const row = template.content.firstElementChild.cloneNode(true);
  const field = function (name) { return row.querySelector('[data-field="' + name + '"]'); };
  const href = 'studioNx.html?projectId=' + item.project_id;
  const bpm = String(Number(item.bpm));             // '128.50' → '128.5', '120.00' → '120'
  const key = item.music_key ? item.music_key : '—';
  const isOwner = item.my_role === 'OWNER';

  row.dataset.projectId = item.project_id;
  row.dataset.title = item.title;                                        // 34 가 수정 칸을 채울 때 씀
  row.dataset.description = item.description ? item.description : '';
  row.querySelector('.pj-row__head').href = href;
  row.querySelector('.pj-row__act a').href = href;
```

```
  // 제목 · 설명 · Not opened
  const title = row.querySelector('.pj-row__head b');
  title.textContent = item.title;
  if (!item.opened) {
    const badge = document.createElement('span');
    badge.className = 'pj-badge';
    badge.textContent = 'Not opened';
    title.appendChild(document.createTextNode(' '));
    title.appendChild(badge);
  }
  row.querySelector('.pj-row__head > span').textContent = item.description ? item.description : 'No desc
ription';

  // BPM · 박자 · 키 · 언제 (모바일 한 줄 정보는 data-meta)
  field('bpm').textContent = bpm;
  field('sig').textContent = item.time_sig_num + '/' + item.time_sig_den;
  field('key').textContent = key;
  const when = field('when');
  when.textContent = formatRelativeTime(item.updated_at);
  when.dataset.meta = bpm + ' BPM · ' + (item.music_key ? item.music_key + ' · ' : '');

  // 참여자 점 (최대 3개) + 내 역할 (OWNER 가 아니면)
  const members = field('members');
  for (let i = 0; i < Math.min(item.member_count, 3); i++) {
    members.appendChild(document.createElement('i'));
  }
  let membersLabel = item.member_count === 1 ? '1 member' : item.member_count + ' members';
  if (!isOwner) {
    const roleName = item.my_role.charAt(0) + item.my_role.slice(1).toLowerCase();   // 'EDITOR' → 'Edit
or'
    const role = document.createElement('span');
    role.className = 'pj-role';
    role.textContent = roleName;
    members.appendChild(role);
    membersLabel += ', your role: ' + roleName.toLowerCase();
  }
  members.setAttribute('aria-label', membersLabel);

  // ··· 메뉴 — 삭제는 OWNER 만, 수정은 OWNER · EDITOR 만
  row.querySelector('.pj-more').setAttribute('aria-label', 'More actions for ' + item.title);
  if (!isOwner) {
    const deleteButton = row.querySelector('[data-action="delete"]');
    deleteButton.disabled = true;
    deleteButton.title = 'Only the owner can delete';
    const small = document.createElement('small');
    small.textContent = 'Owner only';
    deleteButton.appendChild(document.createTextNode(' '));
    deleteButton.appendChild(small);
  }
  if (item.my_role === 'VIEWER') {
    const editButton = row.querySelector('[data-action="edit"]');
    editButton.disabled = true;
    editButton.title = 'Viewers can’t edit';
  }
  return row;
}
```

CSS (`myproject.css`, 이번에 추가)

```
/* JS 가 hidden 으로 숨기는 것 (목록 0개 · 페이지 1개) — display: grid · flex 가 hidden 을 이기지 않게 (P1-14) */
.pj-list [hidden], .pj-pager[hidden] { display: none; }
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
list.replaceChildren(...data.items.map((item) => buildRow(template, item)));
const field = (name) => row.querySelector(`[data-field="${name}"]`);
members.append(...Array.from({ length: Math.min(item.member_count, 3) }, () => document.createElement('i')));
title.append(' ', badge);
```

최신 문법	실제 코드	뜻
`list.replaceChildren(...줄들)`	`list.textContent = ''` + `for` 로 `appendChild`	안을 비우고 새로 채우기
`(name) => …` (화살표 함수)	`function (name) { return …; }`	짧은 함수
`Array.from({ length: n }, () => …)`	`for (let i = 0; i < n; i++)`	n 번 만들기
`el.append(' ', badge)`	`appendChild(createTextNode(' '))` + `appendChild(badge)`	글자와 요소를 한 번에 붙이기

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	같은 파일의 `current` · `formatRelativeTime` 을 그대로 부름

식	뜻	예
`template.content.firstElementChild.cloneNode(true)`	틀 안 첫 요소(`<li>`)를 **안쪽까지 통째로** 복사	`true` 가 없으면 `<li>` 껍데기만
`el.textContent = 값`	글자로만 넣음 — `<img …>` 도 태그가 아니라 글자로 보임	제목 `<img src=x onerror=…>` → 그 글자 그대로
`list.textContent = ''`	안쪽을 모두 지움 (HTML 예시 줄 포함)	
`row.dataset.projectId = 5`	`data-project-id="5"` 속성 (34 · 36 · 37 이 읽음)	
`String(Number('128.50'))`	글자 → 숫자 → 글자, 끝의 0 이 빠짐	`'128.5'`, `'120'`
`item.my_role.charAt(0) + item.my_role.slice(1).toLowerCase()`	첫 글자만 대문자	`'EDITOR'` → `'Editor'`
`'.pj-row__head > span'`	`<a>` **바로 아래**의 span (제목 안 `Not opened` span 은 아님)	설명 칸
`!(isEmpty && !current.query)`	"비었고 검색어도 없음" 이 **아니면** 숨김	

4. 흐름

1. 개수 글자 · 빈 목록 안내 (검색어 유무로 둘 중 하나) · 목록과 제목 줄 보이기/숨기기.
2. 목록 비우기 → 프로젝트마다 `buildRow`: 틀 복사 → 번호 · 링크 → 제목(+ Not opened) · 설명 → BPM · 박자 · 키 · 언제 → 멤버 점 · 역할 → … 메뉴 잠금.

5. 왜 이렇게

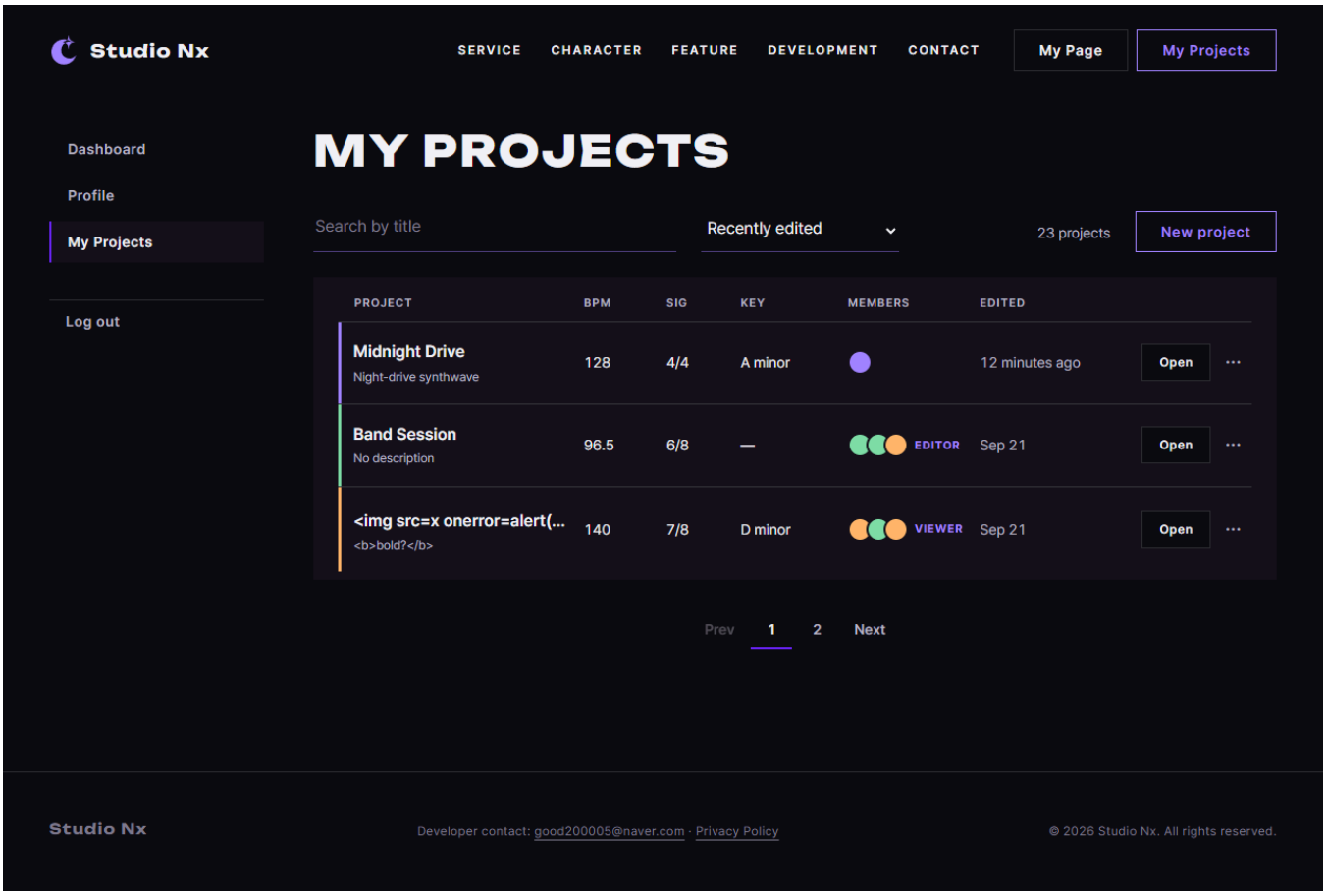
선택	이유
모든 글자를 `textContent` 로	제목 · 설명은 사용자가 친 글자. `innerHTML` 이면 `<img onerror=…>` 같은 제목이 **실행됨** (XSS). 시험에서 그대로 글자로 보이고 이미지가 0개임을 확인
줄 만들기를 `buildRow` 로 나눔	칸이 많아 한 함수에 다 넣으면 길어짐. 36(수정 뒤 그 줄만 다시 그리기)에서도 씀
`Number(bpm)`	서버가 소수를 글자 `'128.50'` 으로 보냄 (P1-13 확인). 화면 예시처럼 `128` · `128.5` 로
개수는 `total`	페이지당 10개라 `items.length` 는 최대 10 — 전체 개수가 사용자가 알고 싶은 것
빈 안내 두 가지	처음 사용자에게는 "만들기" 버튼, 검색 결과가 없을 때는 "검색어를 바꾸라" — 템플릿대로
VIEWER 는 Edit 도 잠금	템플릿에는 Delete 만 있었지만, 서버가 VIEWER 수정을 403 으로 막음(P1-13 60) → 누르고 실패하는 것보다 미리 잠금. Phase 1 은 OWNER 만 생기므로 지금은 보이지 않음
멤버 점 최대 3개	화면 예시 · 템플릿. 정확한 수는 `aria-label(5 members, your role: viewer)`로 화면 낭독기에
`data-title` · `data-description` (34 에서 추가)	34 가 수정 칸을 채울 때 화면 글자(`Not opened` · `No description` 이 섞임) 대신 읽음
CSS `[hidden]` 규칙 추가	`.pj-row` 가 `display: grid` 라 `hidden` 속성을 이겨 제목 줄이 숨지 않았다 — 규칙이 없는 `.pj-row` 는 hidden 이어도 `grid` 로 계산됨을 시험으로 확인. 32 페이지 넘김(`.pj-pager` 는 flex)도 같은 문제라 함께

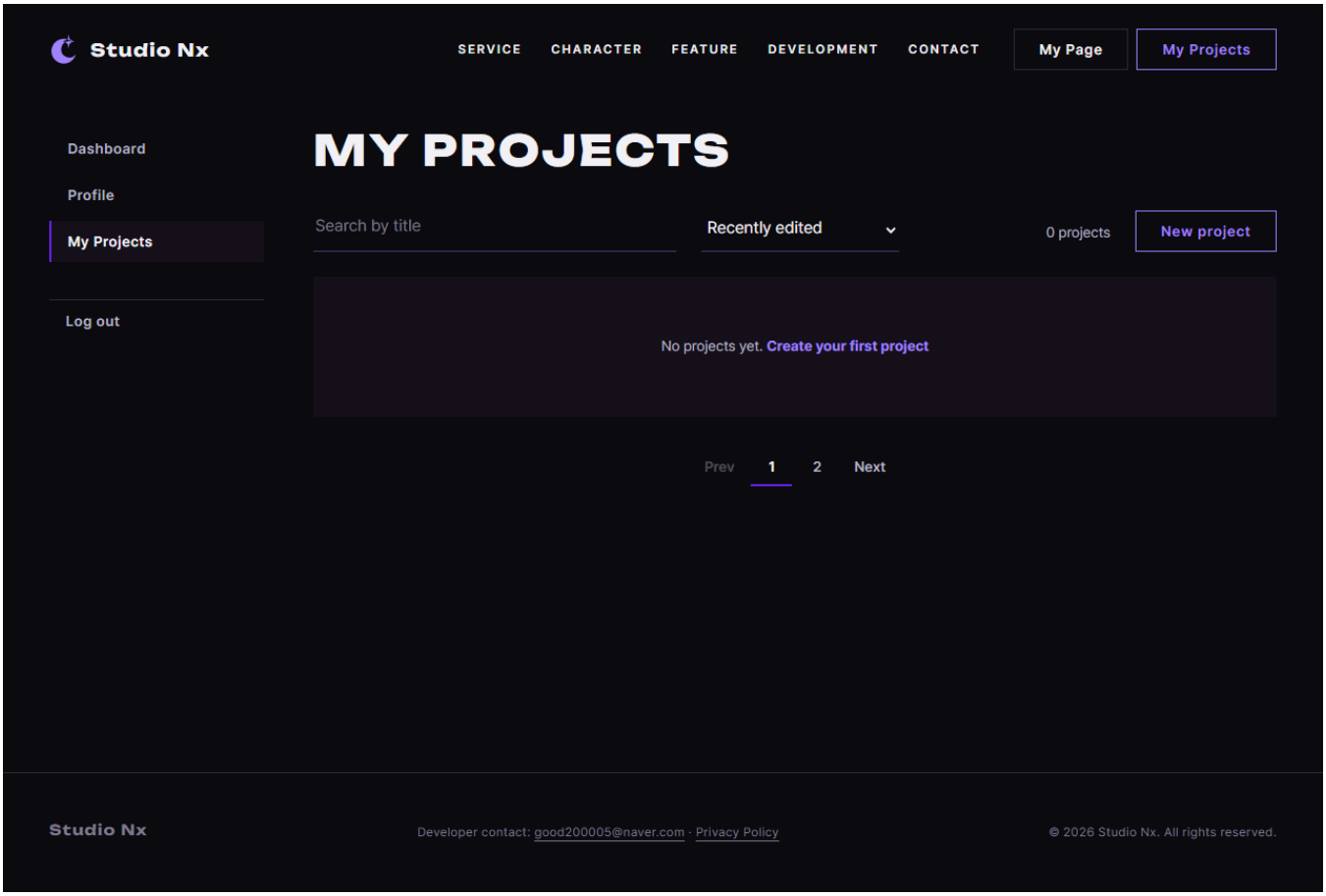
6. 시험 (실제 myProject.html · 실제 서버 · 헤드리스 Chrome)

시험 계정으로 프로젝트 2개를 만들고 실제 목록을 그렸고, 역할 · 멤버 · 키 · XSS 는 가짜 응답으로 그렸다. 보이는지는 **계산된 CSS**(`getComputedStyle(…).display`)로 확인했다. 끝난 뒤 계정 · 프로젝트를 지웠다.

#	상황	결과
01	실제 서버 2개	`2 projects` · 최근 것 먼저 · `Lo-fi Sketch` bpm `128.5` · `7/8` · 키 `–` · `Not opened` / `First Idea` 설명 `No description` · bpm `120` · `Just now` · 링크 `studioNx.html?projectId=…`
02	가짜 3개 (total 23)	`23 projects` / OWNER: 역할 없음 · Delete 열림 · `12 minutes ago` · meta `128 BPM · A minor ·` / EDITOR 3명: 점 3 · `Editor` · Delete 잠금 `Owner only` · Edit 열림 · `Sep 21` / VIEWER 5명: 점 3 · `5 members, your role: viewer` · Edit · Delete 잠금
02	제목 `<img src=x onerror=alert(1)>` · 설명 `<b>bold?</b>`	글자 그대로, 목록 안 이미지 0개
03	0개 · 검색어 없음	`0 projects` · `No projects yet` 보임 · 목록 · 제목 줄 숨김
04	0개 · 검색어 `zzz` (실제)	`No projects match your search.` 보임
05	다시 1개	`1 project` · 목록 · 제목 줄 다시 보임

캡처


Studio Nx
SERVICE
CHARACTER
FEATURE
DEVELOPMENT
CONTACT
My Page
My Projects
Dashboard
Profile
My Projects
Log out
MY PROJECTS
Search by title
Recently edited
23 projects
New project
PROJECT
BPM
SIG
KEY
MEMBERS
EDITED
Midnight Drive
Night-drive synthwave
128
4/4
A minor
12 minutes ago
Open
Band Session
No description
96.5
6/8
—
EDITOR
Sep 21
Open
<img src=x onerror=alert(...
bold?
140
7/8
D minor
VIEWER
Sep 21
Open
Prev
1
2
Next
Studio Nx
Developer contact: good200005@naver.com · Privacy Policy
© 2026 Studio Nx. All rights reserved.


Studio Nx
SERVICE
CHARACTER
FEATURE
DEVELOPMENT
CONTACT
My Page
My Projects
Dashboard
Profile
My Projects
Log out
MY PROJECTS
Search by title
Recently edited
0 projects
New project
No projects yet. Create your first project
Prev
1
2
Next
Studio Nx
Developer contact: good200005@naver.com · Privacy Policy
© 2026 Studio Nx. All rights reserved.

JS 32 — `renderPager()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면 — 페이지 넘김 · PRD 01 FR-123
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	31 `renderProjects()` · 33 `initSearch()`

1. 하는 일

목록 아래 페이지 넘김 줄(`#project-pager`)을 **전체 개수에 맞게 다시 그리고**, 번호를 누르면 그 페이지를 받아 목록과 이 줄을 다시 그린다 (페이지는 새로 열리지 않음).

전체 (10개씩)	지금	화면 (별표 = 지금, (x) = 막힘)
0 · 10개	—	줄 숨김
11개 (2쪽)	1	`Prev(x) 1* 2 Next`
3쪽	3	`Prev 1 2 3* Next(x)`
7쪽	4	`Prev 1 2 3 4* 5 6 7 Next` (7쪽까지는 전부)
20쪽	10	`Prev 1 … 9 10* 11 … 20 Next`
20쪽	4	`Prev 1 2 3 4* 5 … 20 Next` (… 가 한 쪽만 가리면 번호로)
20쪽	20	`Prev 1 … 19 20* Next(x)`

링크 주소에도 검색어 · 정렬이 들어간다 — 검색어 `lo fi` · 이름순이면 2쪽 링크는 `?query=lo+fi&sort=title&page=2`. 그래서 **새 탭으로 열어도** 같은 목록이 나온다 (그 주소를 읽는 일은 38).

2. 언제 쓰나

호출하는 곳	화면
`renderProjects` 다음마다 (페이지 시작 · 검색 · 새 프로젝트 · 삭제 — 33 · 35 · 37 · 38)	`renderPager(data.total, data.page, data.size)`
이 함수 자신	번호를 누른 뒤 새 페이지로 다시 그림

3. 코드

import (`myproject.js` 맨 위 — 30 과 같음, 이번에 추가한 것 없음)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
```

```
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import 를 쓰지 않는다 — 같은 파일의 `PAGE_SIZE` · `current` · `loadProjects`(30) · `renderProjects`(31) 를 쓴다.

함수 (링크 주소 만들기는 `pageHref` 로 나눔)

```
export function renderPager(total, page, size = PAGE_SIZE) {
  const pager = document.getElementById('project-pager');
  if (!pager) return;
  const pages = Math.max(1, Math.ceil(total / size));
  pager.textContent = '';
  pager.hidden = pages <= 1;
  if (pages <= 1) return;

  // 버튼 하나 — 갈 수 없으면 href 없이 aria-disabled
  const addButton = function (text, target, label) {
    const button = document.createElement('a');
    button.className = 'pj-pager__btn';
    button.textContent = text;
    if (label) button.setAttribute('aria-label', label);
    if (target < 1 || target > pages) {
      button.setAttribute('aria-disabled', 'true');
    } else {
      button.href = pageHref(target);
      button.dataset.page = target;
      if (target === page && !label) button.setAttribute('aria-current', 'page');
    }
    pager.appendChild(button);
  };
  const addGap = function () {
    const gap = document.createElement('span');
    gap.className = 'pj-pager__btn';
    gap.setAttribute('aria-hidden', 'true');
    gap.textContent = '…';
    pager.appendChild(gap);
  };

  addButton('Prev', page - 1, 'Previous page');
  for (let n = 1; n <= pages; n++) {
    // 7쪽 이하는 전부, 넘으면 처음 · 끝 · 지금 앞뒤 하나만 (사이는 …)
    const near = n === 1 || n === pages || Math.abs(n - page) <= 1;
    if (pages <= 7 || near) {
      addButton(String(n), n);
    } else if (n === page - 2 || n === page + 2) {
      // … 가 한 쪽만 가리면 (2 · 끝에서 둘째) 그냥 그 번호를 보임
      if (n === 2 || n === pages - 1) addButton(String(n), n);
      else addGap();
    }
  }
  addButton('Next', page + 1, 'Next page');

  // 클릭 → 같은 검색어 · 정렬로 그 페이지를 받아 다시 그림 (페이지는 바뀌지 않음)
  pager.onclick = async function (event) {
    const button = event.target.closest('a[data-page]');
```

```
    if (!button) return;
    event.preventDefault();
    const data = await refreshList({ query: current.query, sort: current.sort, page: button.dataset.page
});
    if (data) window.scrollTo(0, 0);
  };
}

/** 페이지 링크 주소 — 지금 검색어 · 정렬 + 그 페이지 (loadProjects 가 주소창에 남기는 모양과 같음) */
function pageHref(page) {
  const params = new URLSearchParams();
  if (current.query) params.set('query', current.query);
  if (current.sort !== 'updated') params.set('sort', current.sort);
  if (page !== 1) params.set('page', String(page));
  const search = params.toString();
  return search ? '?' + search : location.pathname;
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const addButton = (text, target, label) => { … };
pager.addEventListener('click', async (event) => { … }, { signal });   // 다시 그릴 때 AbortController 로 옛 것 떼기
const { query, sort } = current;
const data = await loadProjects({ query, sort, page: button.dataset.page });
```

최신 문법	실제 코드	뜻
`(a, b) => { … }`	`function (a, b) { … }`	함수
`addEventListener(…, { signal })`	`pager.onclick = …`	다시 그려도 클릭 처리가 하나만 남게
`const { query, sort } = current` + `{ query, sort }`	`{ query: current.query, sort: current.sort }`	칸 꺼내기 · 줄임 속성

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	같은 파일의 함수 · 값을 그대로 부름

식	뜻	예
`Math.ceil(total / size)`	올림 — 남는 것이 있으면 한 쪽 더	11 / 10 = 1.1 → 2
`Math.max(1, …)`	0쪽이 되지 않게	0개 → 1쪽 (숨김)
`Math.abs(n - page) <= 1`	지금 페이지와 1 이하로 떨어진 번호	지금 10 → 9 · 10 · 11
`button.dataset.page = target`	`data-page="2"` — 누를 수 있는 번호에만 붙임	Prev 가 막혔으면 없음
`aria-current="page"`	화면 낭독기 · CSS 가 "지금 페이지" 로 알아봄 (보라 밑줄)	
`aria-disabled="true"`	막힌 버튼 — CSS 가 흐리게 · 눌리지 않게	
`pager.onclick = 함수`	클릭 처리를 **바꿔 끼움** — 몇 번 다시 그려도 하나만	`addEventListener` 는 부를 때마다 하나씩 쌓임

식	뜻	예
`event.target.closest('a[data-page]')`	누른 곳에서 위로 올라가며 번호 버튼 찾기	`…` · 막힌 버튼은 null → 무시
`event.preventDefault()`	링크 기본 동작(페이지 새로 열기)을 막고 JS 로 받아 그림	
`window.scrollTo(0, 0)`	맨 위로 — 페이지 줄은 목록 아래에 있으므로	

4. 흐름

1. 페이지 수 계산 → 1쪽 이하면 줄 숨기고 끝.
2. `Prev` → 번호들 (7쪽 이하면 전부, 넘으면 처음 · 끝 · 지금 앞뒤 + `…`) → `Next`.
3. 누르면: 링크 기본 동작 막기 → 같은 검색어 · 정렬로 그 페이지 받기(30) → 목록(31) · 이 줄 다시 그리기 → 맨 위로.

5. 왜 이렇게

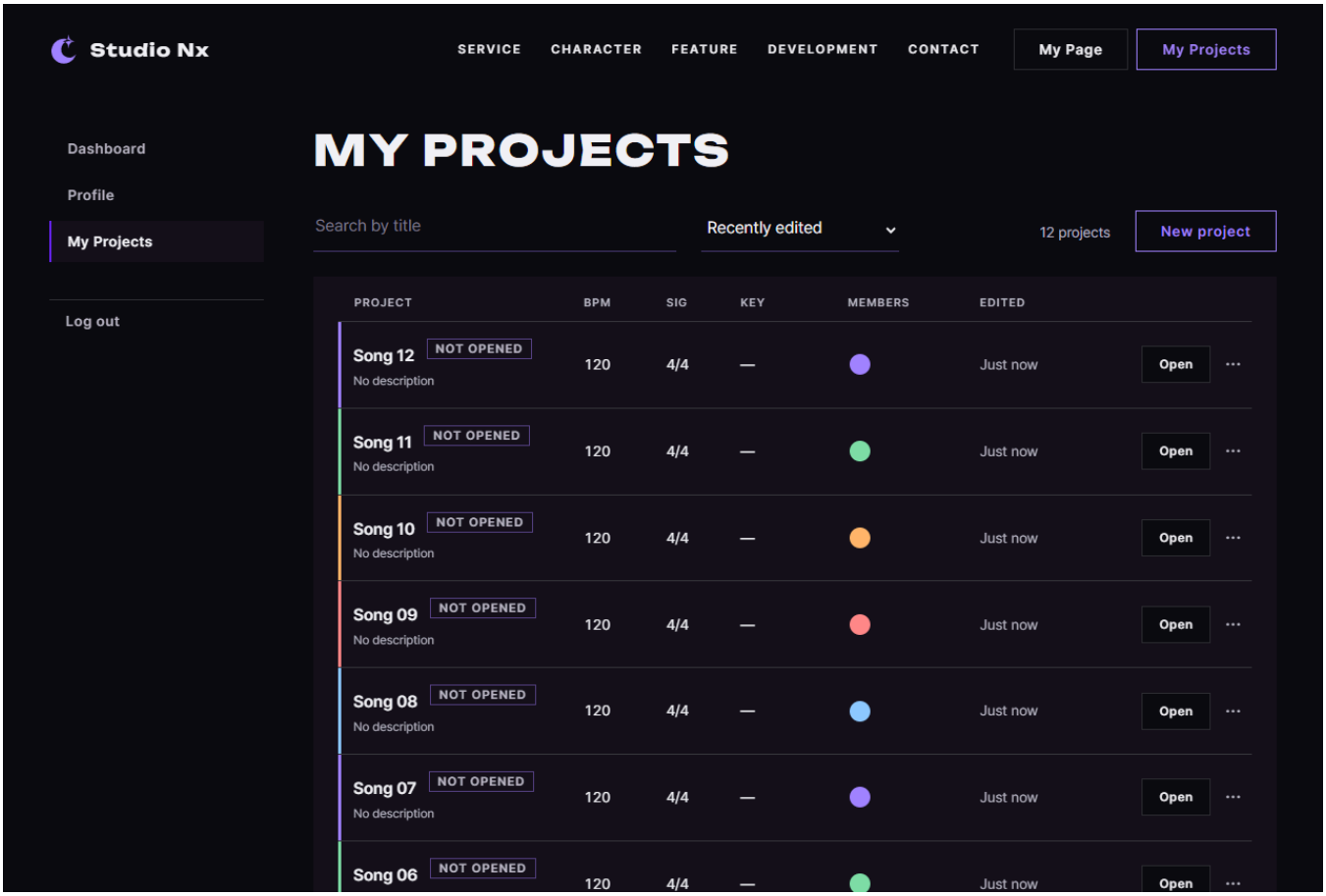
선택	이유
7쪽 넘으면 줄임	프로젝트가 많으면 버튼이 한 줄을 넘침. 처음 · 끝 · 지금 근처만 있으면 어디로든 두세 번에 감
`…` 가 한 쪽만 가리면 번호로	`1 … 3` 의 `…` 는 2 하나뿐 — 그냥 `1 2 3` 이 더 짧고 정확 (처음 만든 뒤 시험에서 보고 고침)
링크에 진짜 주소 (`href`)	마우스 가운데 버튼 · 새 탭으로 열기가 됨. 클릭은 JS 가 가로채 페이지를 새로 열지 않음
막힌 Prev · Next 는 `href` 없음	누를 곳이 아님 — 화면 예시와 같음. CSS 가 흐리게 · 클릭 안 됨
`onclick` 하나	이 함수는 페이지를 넘길 때마다 다시 불림 → `addEventListener` 면 한 번 누를 때 요청이 여러 번. 시험에서 세 번 다시 그린 뒤 한 번 눌러 요청이 **1번**임을 확인
클릭 뒤 처리는 `refreshList` (33 에서 묶음)	받기 → 목록 → 페이지 줄 세 줄이 검색 · 새 프로젝트 · 삭제와 같아서
지금 페이지도 링크	화면 예시와 같음 · 다시 누르면 새로 받기

6. 시험 (실제 myProject.html · 헤드리스 Chrome · 10개)

#	상황	결과
01	전체 0 · 10 · 11개	숨김 · 숨김 · `Prev(x) 1* 2 Next` (숨김은 계산된 CSS 로 확인)
02 · 03	3쪽 중 1 · 3	`Prev(x) 1* 2 3 Next` · `Prev 1 2 3* Next(x)`
04	20쪽 중 10	`Prev 1 … 9 10* 11 … 20 Next`
05	20쪽 중 1 · 3 · 4 · 20	`1* 2 … 20` · `1 2 3* 4 … 20` · `1 2 3 4* 5 … 20` · `1 … 19 20*`
06	8쪽 중 4 · 7쪽 중 4	`1 2 3 4* 5 … 8` · `1 2 3 4* 5 6 7`
07	검색어 `lo fi` · 이름순	링크 `?query=lo+fi&sort=title&page=2` 등 / 기본이면 1쪽 링크 `/myProject.html`
08	실제 프로젝트 12개: 2 누름	목록 2줄 · `Prev 1 2* Next(x)` · 주소 `?page=2` · 요청 **1번** (세 번 다시 그린 뒤)
08	이어서 Prev 누름	목록 10줄 · `Prev(x) 1* 2 Next` · 주소 `myProject.html`
09	막힌 Prev 누름	요청 0번 · 주소 그대로

처음 만든 것은 20쪽 중 4 에서 `1 … 3 4 5` 로 `…` 가 2 하나만 가렸다. 번호로 보이게 고친 뒤 01 ~ 09 를 다시 돌려 모두 통과.

캡처 — 2쪽



JS 33 — `initSearch()` (+ `refreshList`)

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면 — 검색 · 정렬 · PRD 01 FR-123
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	32 `renderPager()` · 34 `initRowMenus()`

1. 하는 일

목록 위 검색 칸 · 정렬 칸이 바뀌면 **1쪽부터 목록을 다시 받아 그린다**.

사용자가 한 일	언제 요청	결과 (주소)
검색 칸에 `alp` 를 침	치기를 **멈추고 0.3초 뒤 한 번**	`alpine, Alpha Song` (`?query=alp`)
공백을 하나 더 침	요청 안 함 (검색어가 같음)	그대로
정렬을 Name 으로	**바로**	`Alpha Song, alpine` (`?query=alp&sort=title`)
검색 칸을 비움	0.3초 뒤	전체 4개 (`?sort=title`)
`gam` 을 치고 바로 Enter	**바로 · 한 번만** (기다리던 것은 취소)	`Gamma Loop`

함께 만든 것

- `refreshList(options)` — "받기(30) → 목록(31) → 페이지 줄(32)" 세 줄을 한 함수로. 32 페이지 넘김도 이것을 쓰도록 바꿨고, 35 · 37 도 쓴다.
- **30 `loadProjects` 에 두 줄 추가 — 늦게 온 옛 응답 버리기.** 아래 §5.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initSearch();` — 검색 칸 · 정렬 칸에 듣기를 한 번 걸어 둠

디바운스(debounce): 글자를 칠 때마다 요청하지 않고, **입력이 멈춘 뒤** 정해진 시간이 지나면 한 번만 하는 방법. 칠 때마다 이전 타이머를 지우고(`clearTimeout`) 새로 건다(`setTimeout`) — 마지막 타이머만 살아남는다. `alp` 를 0.06초 간격으로 치면 요청은 3번이 아니라 1번.

3. 코드

import (`myproject.js` 맨 위 — 30 과 같음, 이번에 추가한 것 없음)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
```

```
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import 를 쓰지 않는다 — 같은 파일의 `current` · `loadProjects` · `renderProjects` · `renderPager` 를 쓴다.

상수 (이번에 추가)

```
/** 검색 칸 입력을 멈춘 뒤 기다리는 시간 (ms) */
const SEARCH_DELAY = 300;

/** loadProjects 요청 번호 — 늦게 도착한 옛 응답을 버리려고 */
let loadTicket = 0;
```

함수

```
export function initSearch() {
  const form = document.getElementById('project-search');
  const input = document.getElementById('project-query');
  const sort = document.getElementById('project-sort');
  if (!form || !input || !sort) return;
  let timer = null;

  const search = function () {
    clearTimeout(timer);
    refreshList({ query: input.value, sort: sort.value, page: 1 });
  };

  input.addEventListener('input', function () {
    clearTimeout(timer);
    if (input.value.trim() === current.query) return;   // 공백만 더 쳤거나 원래대로 되돌림
    timer = setTimeout(search, SEARCH_DELAY);
  });
  sort.addEventListener('change', search);
  form.addEventListener('submit', function (event) {
    event.preventDefault();   // 페이지를 새로 열지 않음
    search();
  });
}

/** 목록 다시 받아 그리기 — loadProjects → renderProjects → renderPager (실패 · 옛 응답이면 아무것도 안 함) */
async function refreshList(options) {
  const data = await loadProjects(options);
  if (!data) return null;
  renderProjects(data);
  renderPager(data.total, data.page, data.size);
  return data;
}
```

30 `loadProjects` 에 추가한 부분

```
  const ticket = ++loadTicket;
  let data;
  try {
```

```
    data = await api.get('/projects?' + params.toString());
  } catch (err) {
    if (ticket !== loadTicket) return null;   // 더 새 요청이 있으면 이 실패는 무시
    …
  }
  if (ticket !== loadTicket) return null;   // 그사이 더 새 요청을 보냄 → 이 응답은 버림 (33)
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
input.addEventListener('input', () => { … });
form.addEventListener('submit', (event) => { event.preventDefault(); search(); });
```

최신 문법	실제 코드	뜻
`() => { … }`	`function () { … }`	함수

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	같은 파일의 함수를 부름

식	뜻	예
`setTimeout(search, SEARCH_DELAY)`	0.3초 뒤 `search` 를 한 번 부름. 번호(타이머)를 돌려줌	
`clearTimeout(timer)`	아직 안 불린 타이머 취소 (이미 지났거나 null 이면 아무 일 없음)	다음 글자를 치면 앞 타이머 취소
`input.value.trim() === current.query`	지금 목록의 검색어와 같으면 요청할 필요 없음	`'alp '` → `'alp'` 같음
`'input'` · `'change'` · `'submit'` 이벤트	글자가 바뀔 때마다 / select 값을 고른 뒤 / Enter (폼 보내기)	
`event.preventDefault()`	폼 기본 동작(주소에 `?query=` 를 붙여 페이지 새로 열기) 막기	
`++loadTicket`	1 늘린 **뒤의** 값 — 요청마다 새 번호	1, 2, 3 …
`ticket !== loadTicket`	내가 보낸 뒤에 다른 요청이 또 나갔는지	내 번호 1, 지금 2 → 옛 것

4. 흐름

- **글자 입력:** 타이머 취소 → 검색어가 지금과 같으면 끝 → 0.3초 타이머.
- **정렬 변경 · Enter:** 타이머 취소 → 바로 `refreshList({ query, sort, page: 1 })`.
- **`refreshList`:** 받기 → 실패 · 옛 응답이면 `null` 로 끝 → 목록 · 페이지 줄 그리기.

5. 왜 이렇게

선택	이유
0.3초 디바운스	템플릿 · HTML 주석의 값. 칠 때마다 요청하면 서버 부담 · 목록이 깜빡임
정렬 · Enter 는 바로	한 번 고르는 동작이라 기다릴 이유가 없음. Enter 는 "지금 찾아" 라는 뜻

선택	이유
검색어가 같으면 건너뜀	공백만 더 쳤거나 지웠다 다시 친 경우 같은 결과를 또 받지 않음
검색 · 정렬은 항상 1쪽부터	3쪽을 보다 검색하면 결과가 1쪽뿐일 수 있음
옛 응답 버리기 (30 에 추가)	`beta` 요청이 늦게 도착하면, 그 뒤에 보낸 `alpine` 결과를 덮어써 화면 · 주소가 `beta` 로 돌아간다. 요청마다 번호를 붙이고 마지막 번호의 응답만 쓴다. **시험에서 이 두 줄을 잠시 빼고 돌리면 실제로 `Beta Beat` 가 남는 것을 확인**하고 되돌렸다
`refreshList` 로 묶음	"받기 → 그리기 → 페이지 줄" 이 32 · 33 · 35 · 37 에서 반복됨
`type="search"` 칸의 × 버튼	누르면 `input` 이벤트가 나서 비운 것과 같게 동작 (따로 처리 없음)

6. 시험 (실제 myProject.html · 실제 서버 · 6개 + 앞 시험 다시)

프로젝트 4개(`Alpha Song`, `Beta Beat`, `Gamma Loop`, `alpine`)를 만들고, 글자 넣기 · 이벤트를 페이지 안에서 일으켰다. `window.fetch` 를 감싸 목록 요청 수를 셌다.

#	한 일	결과
01	`alp` 를 0.06초 간격으로	치는 동안 요청 0 → 멈춘 뒤 **1번** · `alpine, Alpha Song` · `?query=alp`
02	공백 하나 더	요청 0
03	정렬 Name	바로 1번 · `Alpha Song, alpine` · `?query=alp&sort=title`
04	검색 칸 비움	1번 · 4개 · `?sort=title`
05	`gam` + 바로 Enter	0.15초 안에 `Gamma Loop` · 끝까지 요청 **1번** (타이머 취소됨)
06	`beta` 요청을 1.2초 늦게 도착하게 → 그사이 `alpine` + Enter	화면 · 주소 `alpine` 그대로 (옛 응답 버림). 가드를 빼면 `Beta Beat` 로 덮임

앞 시험 다시 (30 · 32 를 고쳤으므로): 30 의 14개, 32 의 10개 모두 그대로 통과 — 페이지 넘김 클릭은 여전히 요청 1번 · `?page=2`.

JS 34 — `initRowMenus()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면 — 줄의 ··· 메뉴 · PRD 01 FR-124 · FR-125
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	33 `initSearch()` · 35 `canOpenStudio()`

1. 하는 일

목록 줄 오른쪽 `···` 메뉴의 **Edit details · Delete 가 어느 프로젝트인지** 기억하고, 모달 칸을 채우고, 메뉴를 닫는다. 모달을 여는 일은 HTML 버튼의 `command="show-modal"` 속성이 이미 한다.

사용자가 한 일	결과
`···` → Edit details	`selected` = 그 프로젝트 · Edit 모달의 제목 · 설명 칸에 지금 값 · 지난 오류 문구 지움 · 메뉴 닫힘
설명 없는 줄의 Edit details	설명 칸은 **빈 칸** (화면 글자 `No description` 이 아님)
`···` → Delete	`selected` = 그 프로젝트 · 메뉴 닫힘 (확인창은 41 `initDeleteProject`)
메뉴를 연 채 다른 줄 `···`	앞 메뉴 닫힘, 새 메뉴만 열림
메뉴 바깥을 누름	열린 메뉴 닫힘
메뉴 안에서 Esc	메뉴 닫힘 · 포커스는 `···` 버튼으로

31 `buildRow` 에 두 줄 추가: 줄에 `data-title` · `data-description` 을 넣는다. 화면의 제목 글자에는 `Not opened`, 설명 글자에는 `No description` 이 섞여 있어서 그대로 읽으면 수정 칸에 잘못 들어가기 때문.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initRowMenus();` — 한 번만
`selected` 를 읽는 곳	36 `initEditProjectForm`(저장할 프로젝트 번호) · 37 `initDeleteProject`(지울 번호 · 확인창 제목)

이벤트 위임(event delegation): 줄마다 버튼에 듣기를 거는 대신 **목록(`ul`) 하나에만** 걸고, 눌린 곳에서 `closest('[data-action]')` 으로 버튼을 찾는 방법. 검색 · 페이지 넘김으로 줄이 다시 그려져도 `ul` 은 그대로라 다시 걸 필요가 없다.

3. 코드

import (`myproject.js` 맨 위 — 30 과 같음, 이번에 추가한 것 없음)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
```

```
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 import 를 쓰지 않는다.

상수 (이번에 추가)

```
/** ··· 메뉴에서 고른 프로젝트 — Edit details · Delete 가 어느 프로젝트인지 (34 가 채움, 36 · 37 이 읽음)
*/
export const selected = { id: null, title: '', description: '' };
```

함수

```
export function initRowMenus() {
  const list = document.getElementById('project-list');
  if (!list) return;

  // ··· 메뉴 안 버튼 — 목록에 하나만 걸어 둠 (줄을 다시 그려도 동작)
  list.addEventListener('click', function (event) {
    const button = event.target.closest('[data-action]');
    if (!button) return;
    const row = button.closest('li[data-project-id]');
    selected.id = Number(row.dataset.projectId);
    selected.title = row.dataset.title;
    selected.description = row.dataset.description;

    if (button.dataset.action === 'edit') {
      const title = document.getElementById('edit-title');
      const description = document.getElementById('edit-description');
      const error = document.getElementById('edit-project-error');
      if (title) title.value = selected.title;
      if (description) description.value = selected.description;
      if (error) { error.hidden = true; error.textContent = ''; }   // 지난번 실패 문구 지우기
    }
    const menu = button.closest('details');
    if (menu) menu.open = false;
  });

  // 메뉴 바깥을 누르면 열린 메뉴 닫기 (다른 줄의 ··· 를 눌러도 앞 메뉴는 닫힘)
  document.addEventListener('click', function (event) {
    const menus = document.querySelectorAll('.pj-menu[open]');
    for (let i = 0; i < menus.length; i++) {
      if (!menus[i].contains(event.target)) menus[i].open = false;
    }
  });

  // Esc → 열린 메뉴 닫고, 메뉴 안에 있던 포커스는 ··· 버튼으로 돌려줌
  document.addEventListener('keydown', function (event) {
    if (event.key !== 'Escape') return;
    const menus = document.querySelectorAll('.pj-menu[open]');
    for (let i = 0; i < menus.length; i++) {
      const hadFocus = menus[i].contains(document.activeElement);
      menus[i].open = false;
      if (hadFocus) menus[i].querySelector('summary').focus();
```

```
    }
  });
}
```

31 `buildRow` 에 추가한 줄

```
  row.dataset.title = item.title;                                    // 34 가 수정 칸을 채울 때 씀
  row.dataset.description = item.description ? item.description : '';
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
for (const menu of document.querySelectorAll('.pj-menu[open]')) {
  if (!menu.contains(event.target)) menu.open = false;
}
row.dataset.description = item.description ?? '';
error?.setAttribute('hidden', '');
```

최신 문법	실제 코드	뜻
`for (const x of 목록)`	`for (let i = 0; i < 목록.length; i++)`	하나씩 돌기
`a ?? ''`	`a ? a : ''`	없으면 빈 글자
`error?.…`	`if (error) …`	있을 때만

문법 풀이

import 문법	뜻
(이 함수는 import 없음)	

식	뜻	예
`event.target.closest('[data-action]')`	누른 곳에서 위로 올라가며 `data-action` 이 있는 요소	버튼 안 글자를 눌러도 버튼을 찾음
`row.dataset.projectId`	`data-project-id` 속성 값 (글자) → `Number(…)` 로 숫자	`'1801'` → 1801
`details.open = false`	`<details>` 메뉴 닫기 (`open` 속성 제거)	
`document.querySelectorAll('.pj-menu[open]')`	열려 있는 메뉴들만	
`menus[i].contains(event.target)`	누른 곳이 그 메뉴 안인지	바깥이면 닫음
`document.activeElement`	지금 포커스가 있는 요소	Esc 전 `Edit details` 버튼

4. 흐름

1. **목록 클릭:** 버튼 찾기 → 그 줄의 번호 · 제목 · 설명을 `selected` 에 → Edit 이면 수정 칸 채우기 · 오류 지우기 / Delete 면 삭제 모달 제목 → 메뉴 닫기.
2. **문서 클릭:** 열린 메뉴 중 누른 곳을 품지 않은 것을 닫음.
3. **Esc:** 열린 메뉴를 닫고, 포커스가 메뉴 안이었으면 `...` 로.

5. 왜 이렇게

선택	이유
목록 하나에 듣기 (위임)	줄은 검색 · 페이지마다 다시 그려짐. 줄마다 걸면 다시 그릴 때마다 다시 걸어야 함 (템플릿)
`data-title` · `data-description` 에서 읽음	화면 글자에서 읽으면 `Neon Rain Not opened` · `No description` 이 수정 칸에 들어감
`selected` 를 밖으로	저장(36) · 삭제(37)는 다른 함수 — 어느 프로젝트인지 넘겨받을 곳이 필요
지난 오류 문구 지움	앞 프로젝트 저장에 실패한 문구가 다른 프로젝트 모달에 남지 않게
바깥 클릭 · Esc 로 닫기	`<details>` 는 다시 눌러야만 닫힘 — 일반 메뉴처럼 동작하게 (템플릿)
Esc 뒤 포커스를 `···` 로	키보드 사용자가 메뉴 안 버튼에 있다가 닫히면 포커스가 사라짐
(41 에서 바꿈) Delete 모달 제목 채우기를 지움	41 이 HTML 삭제 모달을 `confirmDialog` 로 바꿔 모달이 없어짐 — 처음에는 여기서 `#delete-project-name` 을 채웠다

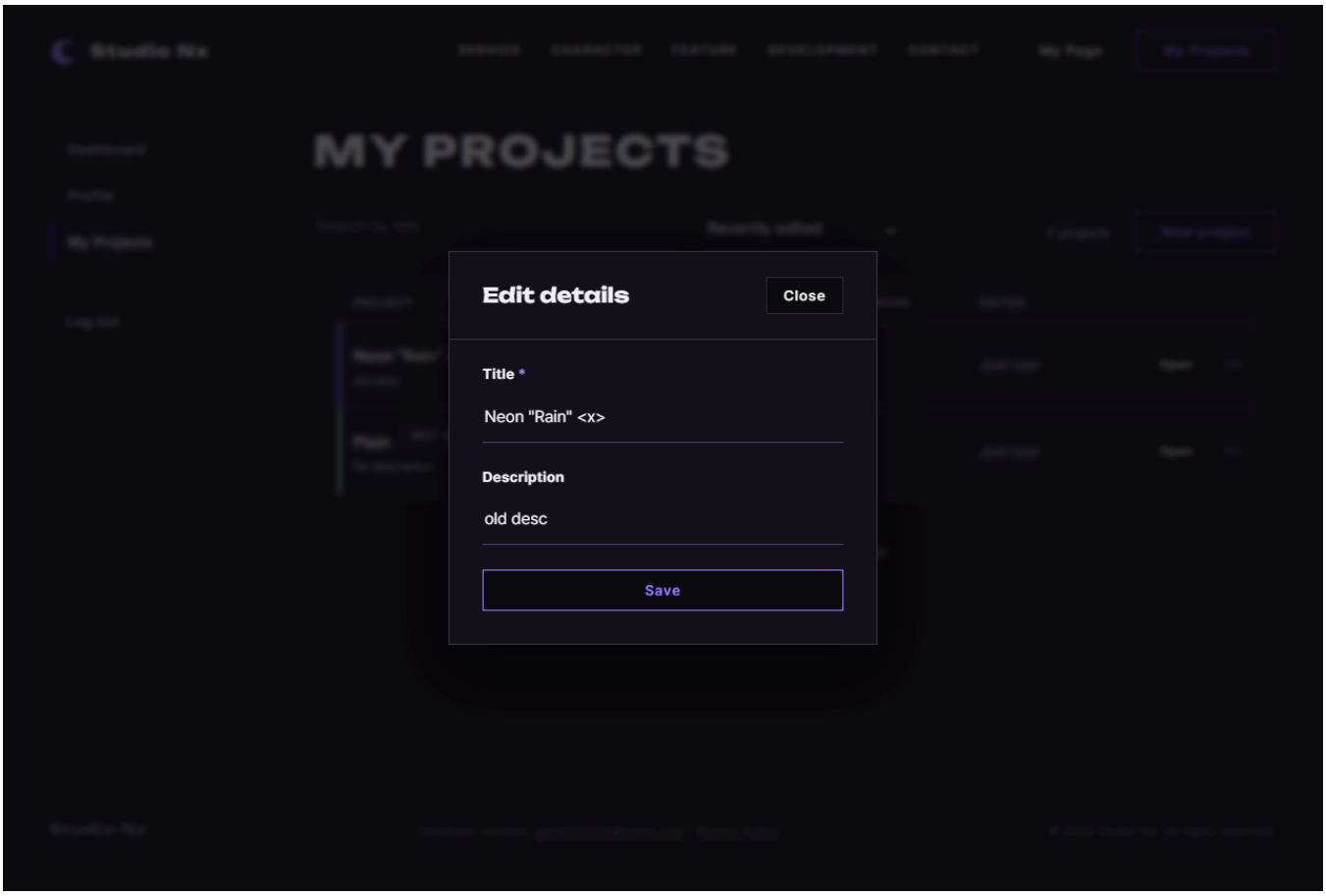
참고: 모달은 버튼의 `command` 속성으로 열린다 (최신 Chrome · Edge 지원). 지원하지 않는 브라우저용 `initModalFallback(core/modal.js)`은 아직 템플릿이다 — 이 작업 범위 밖.

6. 시험 (실제 myProject.html · 실제 서버 · 7개)

프로젝트 `Plain` 과 `Neon "Rain" <x>`(설명 `old desc`)를 만들고 페이지 안에서 눌렀다.

#	한 일	결과
01	첫 줄 `···` → Edit details	메뉴 열림 → 닫힘 · Edit 모달 열림 · 제목 `Neon "Rain" <x>` · 설명 `old desc` · `selected.id` 맞음
02	(오류 문구를 띄워 둔 채) 설명 없는 줄 Edit	설명 칸 빈 칸 · 오류 숨김 · 글자 비움
03	첫 줄 Delete	(41 뒤 다시 시험) 확인창 `Delete project?` · 제목 글자 그대로 · `selected` 맞음 · 메뉴 닫힘
04	첫 메뉴 연 채 둘째 `···`	첫째 닫힘 · 둘째 열림
05	페이지 제목 누름	둘 다 닫힘
06	메뉴 안 `Edit details` 에 포커스 → Esc	닫힘 · 포커스 `···`
07	이름순으로 다시 그린 뒤 Edit	그대로 동작

캡처 — Edit 모달에 채워진 값



JS 35 — `canOpenStudio()` (core/studio.js)

항목	내용
파일	`studionx/frontend/static/js/core/studio.js`
작업	P1-14 내 프로젝트 화면 — 시퀀서를 열 수 있는 화면인지 · PRD 01 FR-129 · PRD 00 반응형 기준
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	34 `initRowMenus()` · 36 `showDesktopOnly()`

1. 하는 일

지금 브라우저 화면 폭이 **1280px 이상**이면 `true`, 아니면 `false`. 시퀀서(`studioNx.html`)는 넓은 화면에서만 열기로 했기 때문 (PRD 00 · 01 FR-129).

화면 폭	결과	예
1920 · 1440 · 1280	`true`	노트북 · 데스크톱
1279 · 1024	`false`	작은 노트북 창 · 태블릿 가로
390	`false`	휴대폰

순서를 바꾼 이유: 새 프로젝트(원래 35)는 이 파일의 `canOpenStudio` · `openStudio` · `showDesktopOnly` 를 쓰는데, `core/studio.js` 네 함수가 모두 아직 템플릿이었다. 이 파일을 처음 쓰는 페이지가 myProject 이므로 여기서 먼저 하나씩 구현한다: **35 `canOpenStudio` → 36 `showDesktopOnly` → 37 `openStudio` → 38 `initDesktopOnlyLinks` → 39 `initNewProjectForm` → 40 수정 → 41 삭제 → 42 페이지 시작.** (myPage 의 `initDesktopOnlyLinks` 도 P1-17 때 그대로 씀)

2. 언제 쓰나

호출하는 곳	화면
39 `initNewProjectForm`	만든 뒤 — 넓으면 시퀀서 창, 좁으면 "컴퓨터에서 여세요" 안내
38 `initDesktopOnlyLinks`	목록의 Open · 제목을 누를 때 (myProject · myPage)

`matchMedia`: CSS 의 `@media (min-width: 1280px)` 조건을 JS 에서 그대로 검사하는 브라우저 기능. `window.innerWidth >= 1280` 과 거의 같지만, 세로 스크롤바 폭 처리까지 **CSS 와 똑같이** 판단한다 — 화면 모양(CSS)과 JS 판단이 1px 차이로 어긋나지 않는다.

3. 코드

import (`studio.js` 맨 위)

```
export const STUDIO_MIN_WIDTH = 1280;
```

`studio.js` 는 다른 파일을 import 하지 않는다. 이 함수는 같은 파일의 상수 `STUDIO_MIN_WIDTH` 를 쓴다.

함수

```
export function canOpenStudio() {
  return window.matchMedia('(min-width: ' + STUDIO_MIN_WIDTH + 'px)').matches;
}
```

최신 문법으로 쓰면 (공부용)

```
export const canOpenStudio = () => matchMedia(`(min-width: ${STUDIO_MIN_WIDTH}px)`).matches;
```

최신 문법	실제 코드	뜻
`` `(min-width: ${STUDIO_MIN_WIDTH}px)` ``	`'(min-width: ' + STUDIO_MIN_WIDTH + 'px)'`	글자 안에 값 넣기
`const f = () => 값`	`function f() { return 값; }`	한 줄 함수

문법 풀이

import 문법	뜻
(import 없음)	`window.matchMedia` 는 브라우저에 원래 있음

식	뜻	예
`window.matchMedia('(min-width: 1280px)')`	그 조건의 검사기	
`.matches`	지금 맞는지 (true / false) — **부를 때마다 새로** 검사	창을 줄인 뒤 부르면 false

4. 흐름

1. `'(min-width: 1280px)'` 조건을 만들어 → 2. 지금 맞는지 돌려준다.

5. 왜 이렇게

선택	이유
`matchMedia` (템플릿)	CSS 반응형 기준과 같은 방식 — 스크롤바 · 확대 처리가 CSS 와 같음
부를 때마다 검사 (값을 저장해 두지 않음)	사용자가 창 크기를 바꿀 수 있음. 버튼을 누른 그 순간의 폭이 기준
CSS 픽셀 기준	고해상도 화면(DPR 2 · 3)도 CSS 폭으로 판단 — 1280 CSS px 이면 실제 픽셀이 2560 이어도 같은 레이아웃. 브라우저 확대(Ctrl +)를 하면 CSS 폭이 줄어 false 가 될 수 있는데, 그만큼 화면이 실제로 좁아진 것이므로 그대로 따름
1280 은 상수 `STUDIO_MIN_WIDTH`	다른 곳(안내 문구 · 시험)에서도 같은 숫자를 씀

6. 시험 (실제 myProject.html · 헤드리스 Chrome · 7개)

한 페이지를 열어 둔 채 **화면 폭만 바꿔 가며** 불렀다 (새로 열지 않음 → 부를 때마다 다시 검사함을 확인).

화면 폭 (DPR)	결과
1920 (1)	`true`
1280 (1)	`true` — 경계 포함
1279 (1)	`false`
1024 (1)	`false`
390 (3, 휴대폰)	`false`
1280 (2, 고해상도)	`true`
1440 (1)	`true` — 다시 넓히면 다시 true

돌려준 값은 모두 `boolean`.

JS 36 — `showDesktopOnly()` (core/studio.js)

항목	내용
파일	`studionx/frontend/static/js/core/studio.js`
작업	P1-14 내 프로젝트 화면 — 작은 화면 안내 모달 · PRD 01 FR-129
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	35 `canOpenStudio()` · 37 `openStudio()`

1. 하는 일

화면이 좁아 시퀀서를 열 수 없을 때(35 가 false) **"컴퓨터에서 여세요" 모달**(`#desktop-only-modal`, `static/partials/desktop-only-modal.html`)을 띄운다. 상황에 맞는 문장 하나만 보이고 제목을 채운다.

부른 값	보이는 문장
`showDesktopOnly('created', 'Summer Loop')`	**Summer Loop** is saved. — 작은 화면에서 프로젝트를 **만든 직후** (39)
`showDesktopOnly('open', 'Summer Loop')`	**Summer Loop** opens in the studio, which needs a larger screen. — Open · 제목을 **눌렀을 때** (38)
그 밖의 값 (`'xyz'`)	`open` 문장

두 경우 모두 아래에 "Sign in on a computer and it'll be waiting at the top of your dashboard." 와 Got it 버튼.

2. 언제 쓰나

호출하는 곳	화면
39 `initNewProjectForm`	작은 화면에서 만들기 성공 → `'created'`
38 `initDesktopOnlyLinks`	작은 화면에서 Open · 제목 클릭 → `'open'` (myProject · myPage)

`<dialog>` 와 `showModal()`: HTML 기본 모달. `showModal()` 로 열면 뒤 화면이 막히고 Esc 로 닫힌다. 이미 열린 모달에 다시 `showModal()` 을 부르면 오류(`InvalidStateError`)가 나므로 `modal.open` 을 먼저 본다.

3. 코드

import (`studio.js` 맨 위)

```
export const STUDIO_MIN_WIDTH = 1280;
```

`studio.js` 는 import 가 없다. 이 함수는 브라우저 기본 기능만 쓴다.

함수

```
export function showDesktopOnly(variant, title) {
  const modal = document.getElementById('desktop-only-modal');
  if (!modal) return;
```

```
  if (variant !== 'created') variant = 'open';

  // 상황에 맞는 문단만 보이기
  const leads = modal.querySelectorAll('[data-variant]');
  for (let i = 0; i < leads.length; i++) {
    leads[i].hidden = leads[i].dataset.variant !== variant;
  }
  const name = document.getElementById('desktop-only-' + variant + '-name');
  if (name) name.textContent = title;

  if (!modal.open) modal.showModal();
}
```

최신 문법으로 쓰면 (공부용)

```
for (const lead of modal.querySelectorAll('[data-variant]')) lead.hidden = lead.dataset.variant !== vari
ant;
document.getElementById(`desktop-only-${variant}-name`)?.replaceChildren(title);
modal.open || modal.showModal();
```

최신 문법	실제 코드	뜻
`for (const x of 목록)`	`for (let i = 0; …; i++)`	하나씩
`el?.replaceChildren(title)`	`if (name) name.textContent = title`	있을 때만 글자 넣기
`a || b()`	`if (!a) b()`	a 가 아니면 b 실행

문법 풀이

import 문법	뜻
(import 없음)	

식	뜻	예
`modal.querySelectorAll('[data-variant]')`	모달 안 `data-variant` 가 있는 문단들	created · open 두 개
`leads[i].hidden = 조건`	조건이 true 면 숨김	고른 것과 다르면 숨김
`'desktop-only-' + variant + '-name'`	고른 문장 안의 제목 칸 id	`desktop-only-created-name`
`name.textContent = title`	글자로만 넣기	`Night <b>Drive</b>` → 그 글자 그대로
`modal.open`	지금 열려 있는지	

4. 흐름

1. 모달이 없으면 끝 → 2. 모르는 값이면 `'open'` → 3. 고른 문장만 보이기 → 4. 제목 넣기 → 5. 닫혀 있으면 열기.

5. 왜 이렇게

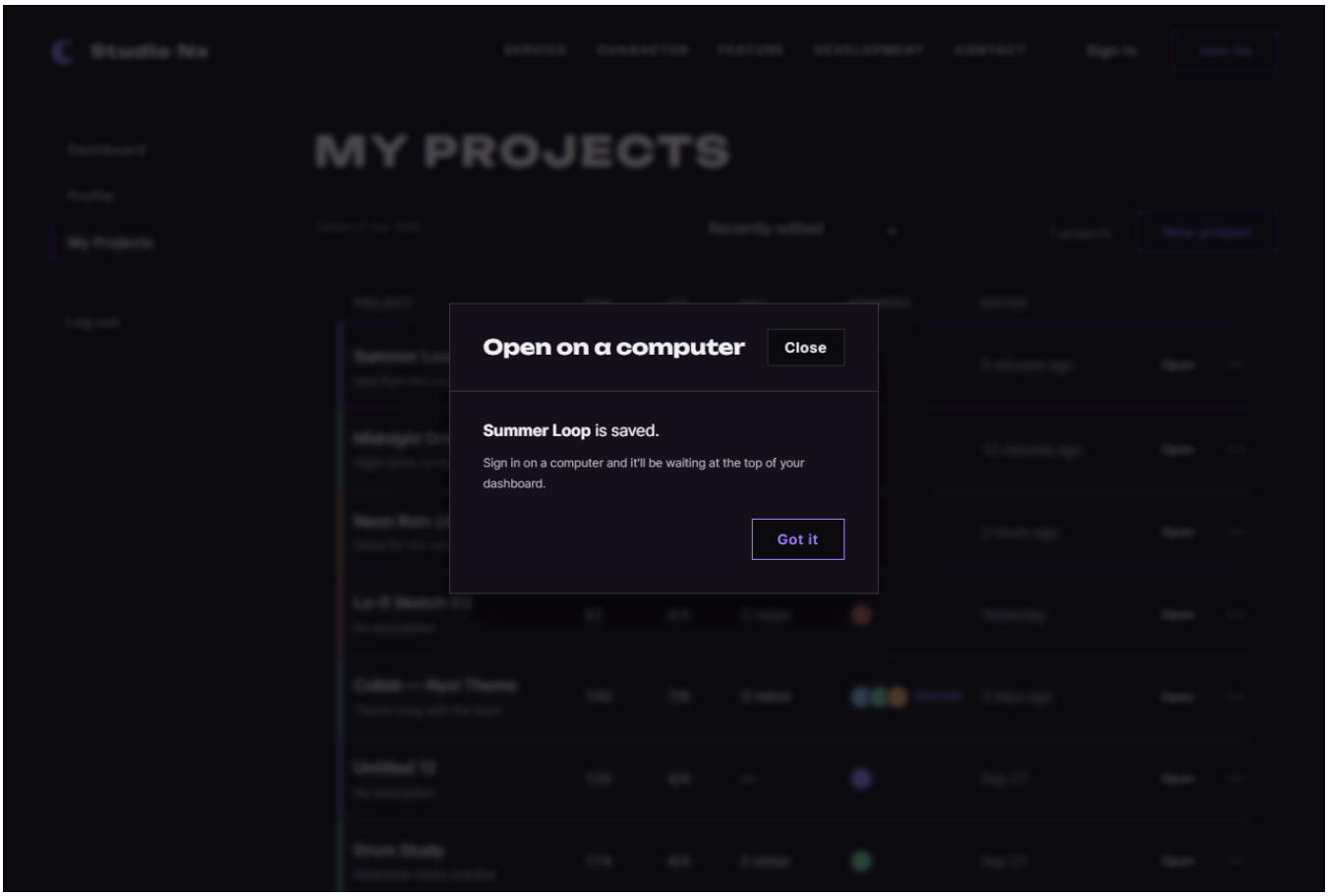
선택	이유
문장 두 개를 HTML 에 두고 숨기기만	글자는 HTML(화면 담당)이 정하고 JS 는 고르기만 — 문구를 고칠 때 JS 를 건드리지 않음 (템플릿)
`textContent`	제목은 사용자가 친 글자 — 태그로 해석되지 않게 (31 과 같음)
모르는 값은 `'open'`	부르는 쪽이 잘못 넘겨도 모달이 빈 문장으로 뜨지 않게
열려 있으면 다시 열지 않음	`showModal()` 을 두 번 부르면 오류. 문장 · 제목만 바꿈
모달이 없으면 조용히 끝	공통 조각(`includePartials`)이 아직 안 끼워진 페이지에서 불려도 오류가 나지 않게

6. 시험 (실제 myProject.html · 헤드리스 Chrome · 5개)

보이는지는 계산된 CSS 로 확인했다.

#	한 일	결과
01	`('created', 'Summer Loop')`	모달 열림 · created 문장만 보임 · 제목 `Summer Loop`
02	닫고 `('open', 'Night <b>Drive</b>')`	open 문장만 · 제목은 글자 그대로 (`<b>` 요소 0개)
03	열린 채 `('created', 'Second')`	오류 없음 · 문장이 created 로 바뀌고 제목 `Second`
04	`('xyz', 'Fallback')`	open 문장 · 제목 `Fallback`
05	Got it 누름	닫힘

캡처 — 만든 직후 안내 (`created`)



JS 37 — `openStudio()` (+ `openBlankStudio`) (core/studio.js)

항목	내용
파일	`studionx/frontend/static/js/core/studio.js`
작업	P1-14 내 프로젝트 화면 — 시퀀서를 별도 창으로 열기 · PRD 01 FR-126 · PRD 02 FR-258 (2026-10-05 결정)
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	36 showDesktopOnly() · 38 initDesktopOnlyLinks()

1. 하는 일

시퀀서(`studioNx.html?projectId=번호`)를 **별도 브라우저 창**으로 연다. 프로젝트 목록 창을 닫아도 작업 창은 남게 하려는 결정(2026-10-05).

부른 값	결과
`openStudio(5)`	화면 크기 창 `studio-5` 에 `studioNx.html?projectId=5`
같은 프로젝트를 또 `openStudio(5)`	**새 창이 아니라** 이미 열린 `studio-5` 창이 앞으로
`openStudio(7, { isNew: true })`	`…?projectId=7&new=1` — 로딩 화면 "새 프로젝트" 모드
`openStudio(9, { win: 빈창 })`	미리 열어 둔 빈 창의 주소를 `…?projectId=9` 로, 이름을 `studio-9` 로
팝업이 막힘 (`window.open` 이 null)	**지금 탭**에서 시퀀서로 이동

함께 만든 것 — `openBlankStudio()`: 새 프로젝트(39)는 서버가 번호를 돌려줘야 주소를 알 수 있다. 그런데 브라우저는 **클릭한 바로 그 순간**(사용자 동작 안)에만 새 창을 허락한다 — 응답을 기다린 뒤 열면 팝업으로 막힌다. 그래서 클릭 때 빈 창을 먼저 열고(`openBlankStudio`), 응답이 오면 `openStudio(id, { win })` 로 주소를 넣는다.

2. 언제 쓰나

호출하는 곳	화면
38 `initDesktopOnlyLinks`	넓은 화면에서 Open · 제목 클릭 → `openStudio(id)` (myProject · myPage)
39 `initNewProjectForm`	Create 클릭 → `openBlankStudio()` → 응답 뒤 `openStudio(id, { isNew: true, win })`, 실패하면 빈 창 닫기

팝업 차단과 사용자 동작(user activation): 브라우저는 클릭 · 키 입력 직후 짧은 동안만 `window.open` 을 허락한다. 한 번 허락받아 창을 열면 그 허락은 **써 버린 것**이라, 같은 클릭으로 창을 또 열면 막힌다 (시험 05 에서 확인).

창 이름: `window.open(주소, 이름)` 에서 같은 이름의 창이 이미 있으면 새로 만들지 않고 그 창을 쓴다. 프로젝트마다 `studio-번호` 로 이름을 붙여 같은 프로젝트가 창 두 개로 열리지 않게 한다 (협업 · 저장 충돌 방지).

3. 코드

import (`studio.js` 맨 위)

```
export const STUDIO_MIN_WIDTH = 1280;
```

`studio.js` 는 import 가 없다. 이 함수는 같은 파일의 `STUDIO_WINDOW_PREFIX('studio-')`를 쓴다.

함수 (`openBlankStudio` · 창 옵션 `studioWindowFeatures` 포함)

```
export function openStudio(projectId, options = {}) {
  let url = 'studioNx.html?projectId=' + encodeURIComponent(projectId);
  if (options.isNew) url += '&new=1';
  const name = STUDIO_WINDOW_PREFIX + projectId;

  // 미리 열어 둔 빈 창이 있으면 그 창에서 (새 프로젝트 – 응답을 기다린 뒤라 새로 열면 팝업으로 막힘)
  let win = options.win;
  if (win && !win.closed) {
    win.name = name;            // 다음에 같은 프로젝트를 누르면 이 창이 앞으로 오게
    win.location.href = url;
  } else {
    win = window.open(url, name, studioWindowFeatures());
  }

  if (!win) {
    location.href = url;        // 팝업이 막힘 → 같은 탭에서 열기
    return null;
  }
  win.focus();
  return win;
}

/**
 * 시퀀서용 빈 창을 먼저 열기 – 클릭 처리 안에서 **await 전에** 부를 것 (39 새 프로젝트)
 * 응답이 오면 openStudio(id, { win }) 로 주소를 넣고, 실패하면 win.close()
 * (구현: AI – 2026-10-08, 37 과 함께)
 * @returns {Window|null}  팝업이 막혔으면 null
 */
export function openBlankStudio() {
  return window.open('', '', studioWindowFeatures());
}

/** 창 옵션 – 'popup' 이 있어야 탭이 아니라 별도 창, 크기는 화면에 맞춤 */
function studioWindowFeatures() {
  return 'popup,width=' + screen.availWidth + ',height=' + screen.availHeight + ',left=0,top=0';
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
export function openStudio(projectId, { isNew = false, win = null } = {}) {
  const url = `studioNx.html?projectId=${encodeURIComponent(projectId)}${isNew ? '&new=1' : ''}`;
  …
  win?.focus();
}
const studioWindowFeatures = () => `popup,width=${screen.availWidth},height=${screen.availHeight},left=0,top=0`;
```

최신 문법	실제 코드	뜻
`{ isNew = false, win = null } = {}` (구조 분해 매개변수)	`options = {}` + `options.isNew` · `options.win`	객체에서 칸 꺼내기

최신 문법	실제 코드	뜻
템플릿 문자열	`'…' + 값 + '…'`	글자 잇기
`win?.focus()`	`if (win) win.focus()`	있을 때만

문법 풀이

import 문법	뜻
(import 없음)	`window.open` · `screen` 은 브라우저에 원래 있음

식	뜻	예
`encodeURIComponent(projectId)`	주소에 넣어도 안전한 글자로	숫자는 그대로
`window.open(url, name, features)`	새 창(또는 같은 이름의 창)을 열고 그 창 객체를 돌려줌. 막히면 `null`	
`'popup,width=…,height=…,left=0,top=0'`	`popup` 이 있으면 탭이 아니라 **별도 창**	
`screen.availWidth` · `availHeight`	작업 표시줄을 뺀 화면 크기	1920 × 1040
`win.closed`	사용자가 그 창을 닫았는지	
`win.name = name`	빈 창에 나중에 이름 붙이기 (같은 사이트 창이라 가능)	`''` → `'studio-9'`
`win.location.href = url`	그 창의 주소를 바꿈	
`location.href = url`	**지금 탭**의 주소를 바꿈	팝업이 막혔을 때

4. 흐름

1. 주소(`studioNx.html?projectId=…` + 새 프로젝트면 `&new=1`)와 창 이름(`studio-…`)을 만든다.
2. 미리 연 빈 창이 있고 아직 열려 있으면 → 그 창에 이름 · 주소. 없으면 → `window.open` 으로 연다.
3. 창을 못 열었으면(막힘) 지금 탭에서 열고 `null`, 열었으면 앞으로 가져오고 그 창을 돌려준다.

5. 왜 이렇게

선택	이유
별도 창 (`popup`)	목록 창을 닫아도 작업 창이 남게 (2026-10-05 결정, PRD 02 FR-258). 새 탭이면 같은 창이라 함께 닫힘
창 이름 `studio-번호`	같은 프로젝트를 다시 누르면 창이 하나 더 생기지 않고 그 창이 앞으로 (시험 02)
빈 창 먼저 (`openBlankStudio`)	새 프로젝트는 응답을 기다려야 번호를 앎 — 기다린 뒤 열면 팝업 차단 (템플릿 설명 그대로)
빈 창에 나중에 이름 붙임	열 때는 번호를 몰라 이름이 없음 → 주소를 넣을 때 `studio-번호` 로 (다음 클릭에서 같은 창을 찾도록)
빈 창을 사용자가 닫았으면 새로 시도	닫힌 창에 주소를 넣을 수 없음. 다만 클릭 허락은 빈 창이 이미 써서 보통 막힌다 → 아래 줄의 같은 탭 열기로 이어짐 (시험 05)
막히면 같은 탭	팝업을 막아 둔 사용자도 시퀀서에는 들어갈 수 있게 (템플릿)
창 크기 = 화면 크기	시퀀서는 넓을수록 좋음 · 1280 이상 화면에서만 열림 (35)

6. 시험 (실제 myProject.html · 헤드리스 Chrome · 6개)

CDP 로 "사용자가 누른 것" 처럼 실행했고(`userGesture`), 열린 창은 Chrome 의 창 목록으로도 확인했다.

#	한 일	결과
01	`openStudio(5)`	`window.open('studioNx.html?projectId=5', 'studio-5', 'popup,width=800,height=600,left=0,top=0')` (시험 화면 800×600) · 창 열림 · 이름 `studio-5` · 창 목록에 추가
02	다시 `openStudio(5)`	**같은 창 객체** · 창 수 그대로
03	`openStudio(7, { isNew: true })`	`?projectId=7&new=1` · `studio-7`
04	`openBlankStudio()` → 0.5초 기다림(응답 흉내) → `openStudio(9, { isNew: true, win })`	빈 창(`about:blank`, 이름 없음)이 **같은 창 그대로** `?projectId=9&new=1` · `studio-9` 로
05	빈 창을 열고 바로 닫음 → `openStudio(11, { win })`	클릭 허락을 빈 창이 이미 써서 새 창은 막힘 → **지금 탭**이 `studioNx.html?projectId=11` 로 이동 (막힘 대비가 동작)
06	`window.open` 이 null 을 돌려주게 함 → `openStudio(3)`	`null` 을 돌려주고 지금 탭이 `studioNx.html?projectId=3` 로

처음엔 05 를 "새 창이 열린다" 고 예상했는데, 실제로는 허락이 이미 쓰여 막히고 같은 탭으로 열렸다. 코드는 그대로 두고(막히면 같은 탭 — 의도한 대비) 설명을 실제 동작에 맞췄다.

JS 38 — `initDesktopOnlyLinks()` (core/studio.js)

항목	내용
파일	`studionx/frontend/static/js/core/studio.js`
작업	P1-14 내 프로젝트 화면 — 시퀀서 링크 클릭 처리 · PRD 01 FR-126 · FR-129
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	37 `openStudio()` · 39 `initNewProjectForm()`

1. 하는 일

목록의 **제목 · Open** 처럼 `studioNx.html` 로 가는 링크를 누르면, 지금 탭에서 넘어가는 대신 화면 폭에 따라 처리한다.

사용자 동작	결과
넓은 화면(1280 이상)에서 **그냥 클릭**	시퀀서가 **새 브라우저 창**으로 (37) — 같은 프로젝트면 열린 창이 앞으로
좁은 화면에서 그냥 클릭	"Open on a computer" 안내 모달 (36) — 제목은 `Not opened` 표 없이
Ctrl · ⌘ · Shift · Alt + 클릭, 휠 클릭	**브라우저 기본 그대로** (새 탭 · 새 창 · 내려받기)
시퀀서가 아닌 링크	그대로

사용자 질문 (2026-10-08): "Ctrl 클릭을 해야 해?" — 아니다. **그냥 클릭 한 번이 새 창**이다. Ctrl+클릭 등은 "새 탭으로 열기" 에 익숙한 사람의 습관을 **막지 않는다**는 뜻이다. 창 / 탭을 사용자가 고르는 설정은 Phase 2 환경 설정(PRD 02 FR-2164)에 있다.

2. 언제 쓰나

호출하는 곳	범위
`myproject.js` 맨 아래 (이미 있음)	`initDesktopOnlyLinks(document.getElementById('project-list'))` — 목록
`mypage.js` (P1-17, 이미 있음)	`initDesktopOnlyLinks(document.getElementById('recent-list'))` — 최근 프로젝트

링크 `href` 는 그대로 둔다 — JS 가 꺼져 있거나 오류가 나도 링크는 지금 탭에서 열린다 (점진적 향상).

3. 코드

import (`studio.js` 맨 위)

```
export const STUDIO_MIN_WIDTH = 1280;
```

`studio.js` 는 import 가 없다. 이 함수는 같은 파일의 `canOpenStudio`(35) · `showDesktopOnly`(36) · `openStudio`(37) 를 쓴다.

함수 (제목 찾기는 `studioLinkTitle` 로 나눔)

```
export function initDesktopOnlyLinks(root = document) {
  if (!root) return;
  root.addEventListener('click', function (event) {
    // 새 탭 · 새 창 · 내려받기 같은 브라우저 기본 동작은 막지 않음
    if (event.defaultPrevented || event.button !== 0) return;
    if (event.ctrlKey || event.metaKey || event.shiftKey || event.altKey) return;
    const link = event.target.closest('a[href^="studioNx.html"]');
    if (!link) return;

    event.preventDefault();
    if (!canOpenStudio()) {
      showDesktopOnly('open', studioLinkTitle(link));
      return;
    }
    const params = new URLSearchParams(link.getAttribute('href').split('?')[1]);
    openStudio(params.get('projectId'));
  });
}

/** 링크가 속한 프로젝트의 제목 — 줄의 data-title, 없으면 <b> 안 첫 글자 마디 */
function studioLinkTitle(link) {
  const row = link.closest('[data-title]');
  if (row) return row.dataset.title;
  const holder = link.querySelector('b') ? link : link.closest('li, section, article, div');
  const bold = holder ? holder.querySelector('b') : null;
  return bold && bold.firstChild ? bold.firstChild.textContent.trim() : 'This project';
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
if (event.ctrlKey || event.metaKey || event.shiftKey || event.altKey) return;   // 같음
const projectId = new URL(link.href).searchParams.get('projectId');
return link.closest('[data-title]')?.dataset.title
    ?? holder?.querySelector('b')?.firstChild?.textContent.trim()
    ?? 'This project';
```

최신 문법	실제 코드	뜻
`a?.b` 사슬	`a ? a.b : null` · `if`	있을 때만 다음으로
`x ?? y`	`x ? x : y`	없으면 다음 값
`new URL(link.href).searchParams`	`new URLSearchParams(href.split('?')[1])`	주소의 `?` 뒤 값 읽기

문법 풀이

import 문법	뜻
(import 없음)	

식	뜻	예
`event.button !== 0`	왼쪽 버튼이 아님 (1 = 휠, 2 = 오른쪽)	휠 클릭은 브라우저가 새 탭
`event.ctrlKey` · `metaKey` · `shiftKey` · `altKey`	누른 채 클릭한 키 (meta = Mac ⌘)	Ctrl = 새 탭, Shift = 새 창, Alt = 내려받기
`event.defaultPrevented`	앞에서 이미 다른 코드가 막았는지	

식	뜻	예
`'a[href^="studioNx.html"]'`	`href` 가 `studioNx.html` 로 **시작하는** 링크	
`link.getAttribute('href').split('?')[1]`	`?` 뒤 부분	`'projectId=7'`
`params.get('projectId')`	그 값	`'7'`
`link.closest('[data-title]')`	위로 올라가며 `data-title` 이 있는 요소 (myProject 줄, 31)	
`bold.firstChild.textContent`	`<b>` 의 첫 글자 마디만 — 뒤의 `<span>Not opened</span>` 는 빠짐	`Night Drive`

4. 흐름

1. 범위(`root`) 안 클릭 → 막힌 클릭 · 왼쪽 버튼이 아님 · 보조 키 → 그대로 둠.
2. 시퀀서 링크가 아니면 그대로 → 맞으면 기본 이동을 막음.
3. 좁은 화면 → 안내 모달(제목 찾기) / 넓은 화면 → 주소의 `projectId` 로 새 창.

5. 왜 이렇게

선택	이유
그냥 클릭 = 새 창	목록 창을 닫아도 작업 창이 남게 (2026-10-05 결정)
보조 키 · 휠은 그대로	사용자가 **직접 고른** 동작(새 탭 등)을 JS 가 바꾸면 불편 — 템플릿의 Ctrl · ⌘ · 휠에 Shift · Alt 도 더함
범위(`root`) 하나에 듣기	목록이 다시 그려져도 동작 (34 와 같은 위임)
제목은 `data-title` 먼저	화면 글자에는 `Not opened` 가 섞임 (31) — myPage 처럼 없으면 `<b>` 의 첫 마디
제목을 못 찾으면 `This project`	모달이 빈 이름으로 뜨지 않게
`href` 를 남겨 둠	JS 없이도 링크가 동작 · 마우스를 올리면 주소가 보임

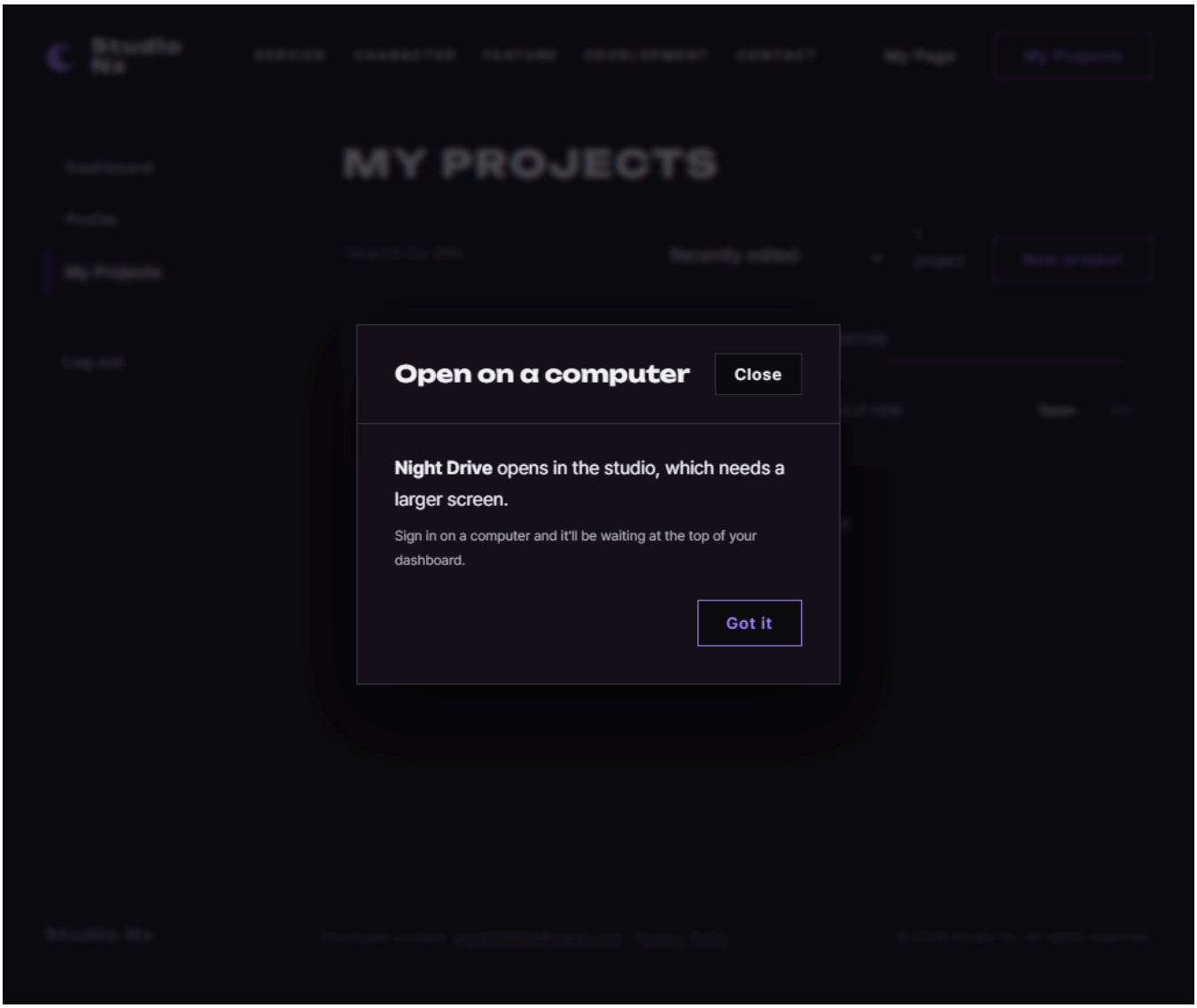
P1-17 참고: myPage 의 'Ready to start' 배너는 제목이 `<h2>` 라 지금 규칙으로는 `This project` 가 된다. 배너는 `recent-list` 밖이라 지금은 이 함수 범위가 아니며, P1-17 에서 배너에 `data-title` 을 넣으면 그대로 맞는다.

6. 시험 (실제 myProject.html · 실제 서버 · 헤드리스 Chrome · 6개)

프로젝트 `Night Drive` 를 만들어 목록을 그리고, "사용자가 누른 것" 으로 클릭했다. 클릭이 막혔는지는 문서 끝에서 `defaultPrevented` 로 기록했다.

#	한 일	결과
01	1280 화면 · 제목 그냥 클릭	막음 · **새 창** `studioNx.html?projectId=…` 생김 · 지금 탭은 myProject 그대로
02	같은 줄 Open 버튼	막음 · 창 수 그대로 (같은 창이 앞으로)
03	Ctrl · Shift · Alt · Meta · 휠 클릭	5번 모두 **안 막음** · 모달 안 뜸
04	1024 화면 · 제목 클릭	막음 · 안내 모달 · 제목 `Night Drive` (`Not opened` 없음) · 새 창 없음
05	`data-title` 없는 myPage 모양 줄	안내 모달 제목 `Midnight Drive`
06	목록 안 다른 링크(`userInfo.html`)	안 막음 · 모달 안 뜸

캡처 — 좁은 화면(1024)에서 제목을 눌렀을 때



JS 39 — `initNewProjectForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면 — 새 프로젝트 만들기 · PRD 01 FR-122 · FR-126 · FR-129
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	38 `initDesktopOnlyLinks()` · 40 `initEditProjectForm()`

1. 하는 일

New project 모달에서 **Create project** 를 누르면 서버에 프로젝트를 만들고(`POST /projects`, P1-13 59), 화면 폭에 따라 다음으로 이어간다.

상황	결과
넓은 화면 (1280 이상)	누른 순간 **빈 새 창** → 응답이 오면 그 창이 `studioNx.html?projectId=…&new=1` (시권서 "새 프로젝트" 로딩)
좁은 화면	창 없이 **"... is saved."** 안내 (36 `created`)
둘 다	모달 닫힘 · 폼 비움 · 검색 · 정렬을 기본으로 되돌리고 목록을 다시 받음 → **새 프로젝트가 맨 위**
제목이 공백뿐	보내지 않고 `Enter a title.`
BPM 10 (20~999 밖) 등	브라우저 검사에서 멈춤 → 알림창 (FIX-01 `initFormAlerts`) · 보내지 않음
서버 오류	미리 연 빈 창을 **닫고** 모달 안에 서버 문구 · 입력값은 그대로
보내는 동안	Create 버튼 잠금 (두 번 눌러 두 개 만들어지지 않게)

보내는 값: `{ title, description(없으면 null), bpm(숫자), time_sig_num, time_sig_den }` — 박자 칸의 `'7/8'` 을 7 과 8 로 나눔.

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initNewProjectForm();` — 폼에 한 번 걸어 둠

모달은 `New project` 버튼의 `command="show-modal"` 이 연다 (HTML). 이 함수는 **보내기**만 맡는다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio, openBlankStudio } from '../co
```

```
re/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

`openBlankStudio` 를 이번에 추가했다. 이 함수는 `api`(P1-02) · `canOpenStudio`(35) · `showDesktopOnly`(36) · `openStudio` · `openBlankStudio`(37) 와 같은 파일의 `refreshList`(33) 를 쓴다.

함수

```
export function initNewProjectForm() {
  const form = document.getElementById('new-project-form');
  if (!form) return;
  const modal = document.getElementById('new-project-modal');
  const error = document.getElementById('new-project-error');
  const submit = form.querySelector('[type="submit"]');
  const showError = function (message) {
    error.textContent = message;
    error.hidden = false;
  };

  form.addEventListener('submit', async function (event) {
    event.preventDefault();
    const title = document.getElementById('new-title').value.trim();
    const description = document.getElementById('new-description').value.trim();
    const sig = document.getElementById('new-sig').value.split('/');   // '7/8' → ['7', '8']
    error.hidden = true;
    if (!title) {
      showError('Enter a title.');
      return;
    }

    // 넓은 화면: 클릭한 지금 빈 창을 열어 둠 (응답 뒤에 열면 팝업으로 막힘 — 37)
    const wide = canOpenStudio();
    const win = wide ? openBlankStudio() : null;
    submit.disabled = true;
    let project;
    try {
      project = await api.post('/projects', {
        title: title,
        description: description ? description : null,
        bpm: Number(document.getElementById('new-bpm').value),
        time_sig_num: Number(sig[0]),
        time_sig_den: Number(sig[1]),
      });
    } catch (err) {
      if (win) win.close();
      showError(err.message);
      return;
    } finally {
      submit.disabled = false;
    }

    // 성공 — 모달 닫고 폼 비우기
    if (modal && modal.open) modal.close();
    form.reset();
    if (wide) {
```

```
      openStudio(project.project_id, { isNew: true, win: win });
    } else {
      showDesktopOnly('created', project.title);
    }

    // 목록은 기본 조건(최근 수정순 · 검색 없음)으로 — 새 프로젝트가 맨 위
    const query = document.getElementById('project-query');
    const sort = document.getElementById('project-sort');
    if (query) query.value = '';
    if (sort) sort.value = 'updated';
    refreshList({ query: '', sort: 'updated', page: 1 });
  });
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const [num, den] = document.getElementById('new-sig').value.split('/').map(Number);
project = await api.post('/projects', { title, description: description || null, bpm: +bpmField.value, t
ime_sig_num: num, time_sig_den: den });
win?.close();
openStudio(project.project_id, { isNew: true, win });
```

최신 문법	실제 코드	뜻
`const [a, b] = 배열`	`sig[0]` · `sig[1]`	배열에서 꺼내기
`.map(Number)`	`Number(sig[0])` 따로	모두 숫자로
`{ title }` · `{ win }`	`{ title: title }` · `{ win: win }`	줄임 속성
`x || null`	`x ? x : null`	비었으면 null
`win?.close()`	`if (win) win.close()`	있을 때만

문법 풀이

import 문법	뜻
`import { …, openBlankStudio } from '../core/studio.js'`	클릭 때 빈 시퀀서 창을 미리 여는 함수 (37)

식	뜻	예
`form.addEventListener('submit', async function (event) { … })`	Enter · Create 를 누르면 (브라우저 검사를 통과한 뒤에만 불림)	
`'7/8'.split('/')`	`/` 로 나눔	`['7', '8']`
`Number('128.5')`	글자 → 숫자	128.5
`try { … } catch { … } finally { … }`	실패하면 catch, **성공 · 실패 모두** 끝에 finally	버튼 잠금 풀기
`catch` 안의 `return` 뒤에도 `finally` 실행		
`form.reset()`	칸을 HTML 처음 값으로 (제목 빈 칸 · BPM 120 · 4/4)	
`modal.close()`	`<dialog>` 닫기	

4. 흐름

1. 기본 보내기(페이지 이동) 막기 → 제목 공백이면 문구 후 끝.
2. 넓은 화면이면 **지금** 빈 창 열기 (기다리기 전).
3. 버튼 잠금 → `POST /projects`.
4. 실패: 빈 창 닫기 · 문구 · 버튼 풀기.
5. 성공: 모달 닫기 · 폼 비우기 → 넓으면 빈 창을 시켠서로, 좁으면 "saved" 안내 → 검색 · 정렬 칸을 기본으로 → 목록 다시 받기(33 `refreshList`).

5. 왜 이렇게

선택	이유
빈 창을 **await 전에**	응답 뒤에 열면 팝업 차단 (37)
실패하면 빈 창 닫기	빈 흰 창이 남지 않게 (시험 05 에서 창 수가 그대로임을 확인)
공백 제목은 보내지 않음	`required` 는 공백 한 칸도 통과시킴 → 서버 422 의 영어 검사 문구보다 짧고 분명하게
검사는 브라우저에 맡김 (BPM 범위 · 필수)	HTML 의 `min` · `max` · `required` 가 이미 있고, FIX-01 이 말풍선 대신 알림창으로 보여 줌
만든 뒤 검색 · 정렬을 기본으로	검색 중(`zzz`)이었다면 새 프로젝트가 목록에 안 보임 → "맨 위에 보인다" (템플릿) 를 지키려고. 검색 칸 · 정렬 칸 글자도 함께 비움
좁은 화면은 "saved" 안내	시켠서를 못 여는 화면 — 만들어졌다는 것과 컴퓨터에서 열면 된다는 것을 알려 줌 (FR-129)
목록 창은 그대로	새 창으로 시켠서가 열리고, 이 창은 갱신된 목록으로 남음 (템플릿)

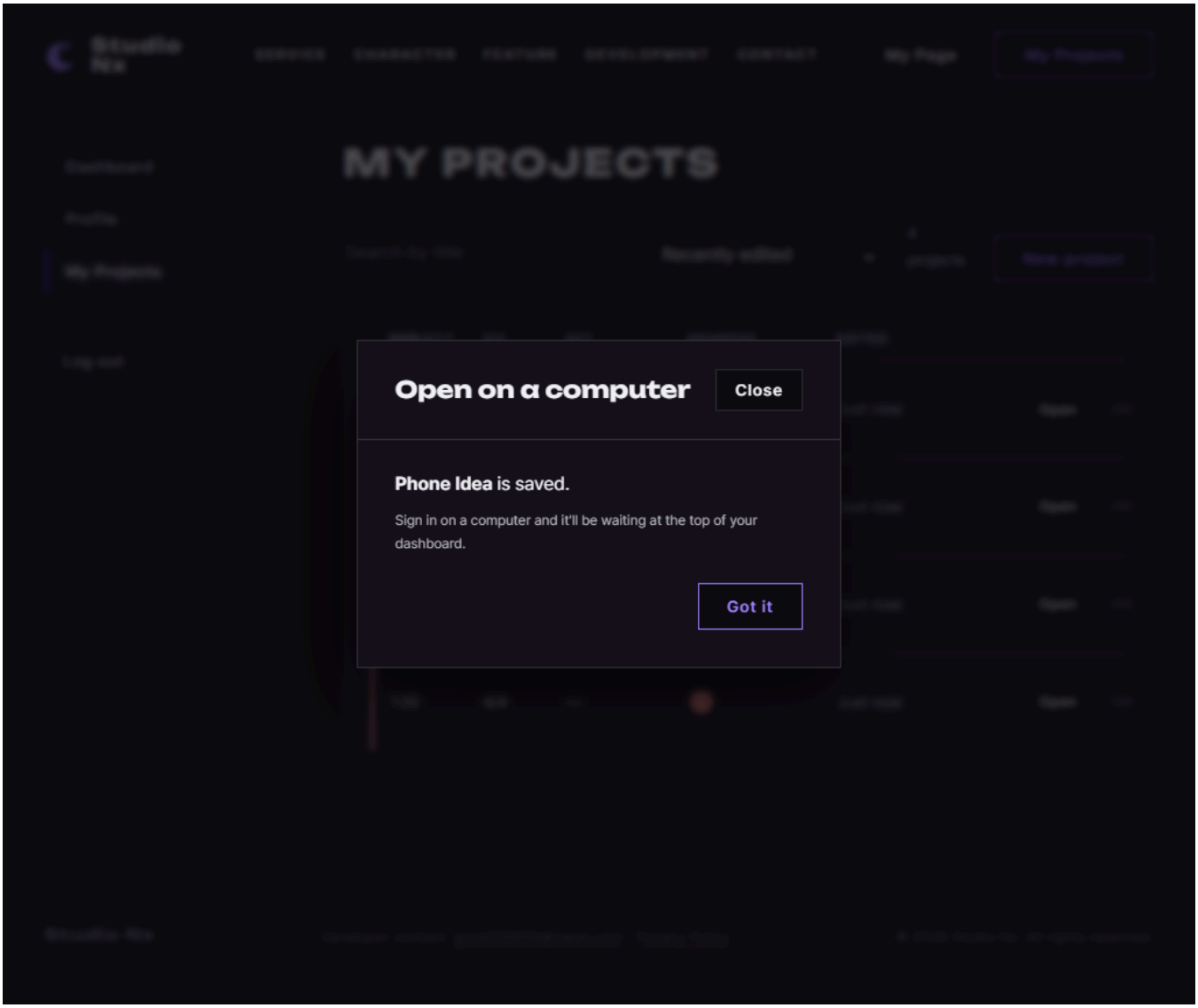
6. 시험 (실제 myProject.html · 실제 서버 · 헤드리스 Chrome · 7개)

New project 버튼으로 모달을 열고 칸을 채운 뒤 보냈다 ("사용자가 누른 것" 으로). 열린 창은 Chrome 창 목록으로 확인했다.

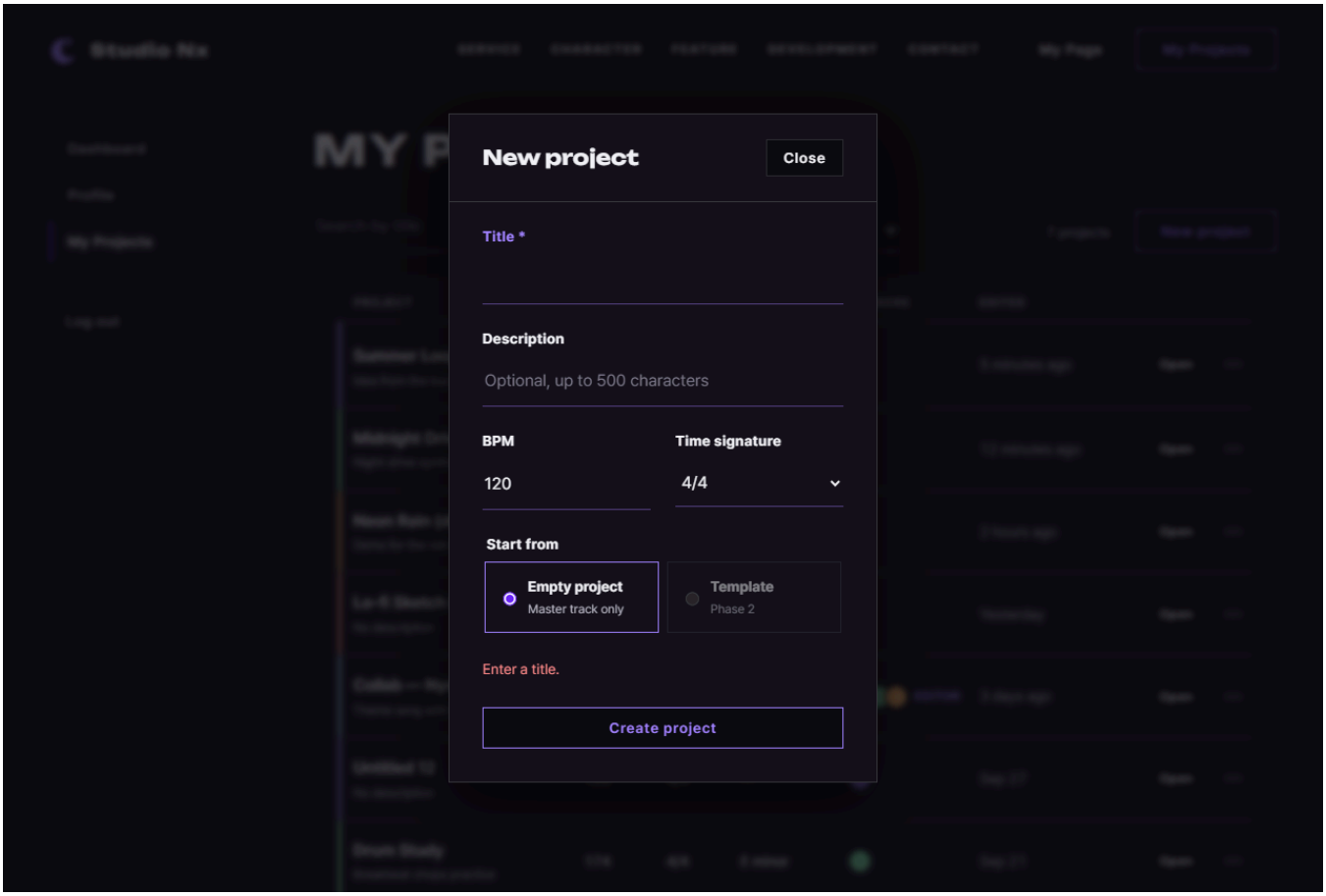
#	한 일	결과
01	1280 · `' New Wave '` · 설명 · 128.5 · 7/8	새 창 `studioNx.html?projectId=…&new=1` · 모달 닫힘 · 제목 칸 비움 · 목록 맨 위 `New Wave` · 서버 저장값 제목 `New Wave` · 설명 · bpm `128.50` · `7/8`
02	검색 `zzz` (0개) 중에 `Second` 만들기	목록이 기본으로 → `Second, New Wave, Old One` · 주소 `?query=zzz` → 없음 · 검색 칸 비움
03	1024 · `Phone Idea` · 3/4	새 창 없음 · "Phone Idea is saved." 안내 · 목록 맨 위
04	제목 공백	`Enter a title.` · 요청 0 · 창 없음 · 모달 그대로
05	서버 500 (가짜)	문구 `Something went wrong. Please try again.` · 빈 창 닫혀 창 수 그대로 · 버튼 풀림 · 입력값 그대로
06	응답을 0.8초 늦춤	보내는 동안 버튼 잠김 → 끝나면 풀림 · 요청 1번
07	BPM 10	요청 0 · 알림창(FIX-01)

캡처

좁은 화면에서 만든 뒤:


MY PROJECTS
Open on a computer
Close
Phone Idea is saved.
Sign in on a computer and it'll be waiting at the top of your dashboard.
Got it

공백 제목:


New project
Close
Title *
Description
Optional, up to 500 characters
BPM
120
Time signature
4/4
Start from
Empty project
Master track only
Template
Phase 2
Enter a title.
Create project

JS 40 — `initEditProjectForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면 — 제목 · 설명 수정 · PRD 01 FR-124
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	39 `initNewProjectForm()` · 41 `initDeleteProject()`

1. 하는 일

Edit details 모달의 **Save** 를 누르면 `PATCH /projects/{id}`(P1-13 60)로 제목 · 설명을 저장하고, 성공하면 **그 줄만** 새 값으로 다시 그린다.

상황	결과
제목 `' Beta v2 '` · 설명 `new`	모달 닫힘 · 그 줄이 `Beta v2 · new` 로 (자리 그대로) · 다른 줄은 그대로
설명을 비우거나 공백만	설명 지움 → 줄에 `No description` · 서버 값 `null`
제목 공백	보내지 않고 `Enter a title.`
그사이 다른 창에서 지워진 프로젝트	`Project not found.` + 목록을 다시 받아 그 줄이 사라짐
VIEWER (31 에서 Edit 이 잠겨 있지만 혹시 보내면)	서버 문구 `You don't have permission.`
서버 오류	모달 안에 문구 · 입력값 그대로 · 버튼 풀림

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initEditProjectForm();`
함께 쓰는 것	34 `initRowMenus` 가 `selected` 와 칸 값을 채우고 모달을 엶 → 이 함수는 저장만

3. 코드

import (`myproject.js` 맨 위 — 39 와 같음, 이번에 추가한 것 없음)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio, openBlankStudio } from '../co
re/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

이 함수는 `api`(P1-02) 와 같은 파일의 `selected`(34) · `current`(30) · `buildRow`(31) · `refreshList`(33) 를 쓴다.

함수

```
export function initEditProjectForm() {
  const form = document.getElementById('edit-project-form');
  if (!form) return;
  const modal = document.getElementById('edit-project-modal');
  const error = document.getElementById('edit-project-error');
  const submit = form.querySelector('[type="submit"]');
  const showError = function (message) {
    error.textContent = message;
    error.hidden = false;
  };

  form.addEventListener('submit', async function (event) {
    event.preventDefault();
    if (selected.id === null) return;
    const title = document.getElementById('edit-title').value.trim();
    const description = document.getElementById('edit-description').value.trim();
    error.hidden = true;
    if (!title) {
      showError('Enter a title.');
      return;
    }

    submit.disabled = true;
    let item;
    try {
      item = await api.patch('/projects/' + selected.id, {
        title: title,
        description: description ? description : null,   // 비우면 설명 지우기
      });
    } catch (err) {
      showError(err.message);
      if (err.status === 404) refreshList({ query: current.query, sort: current.sort, page: current.page
});
      return;
    } finally {
      submit.disabled = false;
    }

    if (modal && modal.open) modal.close();
    selected.title = item.title;
    selected.description = item.description ? item.description : '';
    const row = document.querySelector('#project-list > li[data-project-id="' + item.project_id + '"]');
    const template = document.getElementById('project-row-template');
    if (row && template) row.replaceWith(buildRow(template, item));
  });
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
item = await api.patch(`/projects/${selected.id}`, { title, description: description || null });
selected.description = item.description ?? '';
document.querySelector(`#project-list > li[data-project-id="${item.project_id}"]`)?.replaceWith(buildRow
(template, item));
```

최신 문법	실제 코드	뜻
템플릿 문자열	`'/projects/' + selected.id`	글자 잇기
`a ?? ''`	`a ? a : ''`	없으면 빈 글자
`el?.replaceWith(…)`	`if (row && template) row.replaceWith(…)`	있을 때만

문법 풀이

import 문법	뜻
(새 import 없음)	`api.patch(주소, 본문)` — PATCH 요청, 토큰 자동 (P1-02)

식	뜻	예
`description ? description : null`	빈 글자면 null — 서버가 설명을 지움 (P1-13 56 에서 `""` 도 None 이지만 뜻을 분명히)	
`'li[data-project-id="' + id + '"]'`	그 번호의 줄 찾기	
`row.replaceWith(새 요소)`	그 자리에 새 줄을 끼우고 옛 줄은 뺌	순서 · 다른 줄은 그대로
`err.status === 404`	프로젝트가 없음 (지워졌거나 권한 잃음, P1-13 57)	

4. 흐름

1. 기본 보내기 막기 → 고른 프로젝트가 없으면 끝 → 제목 공백이면 문구.
2. 버튼 잠금 → `PATCH /projects/{id}` `{ title, description }`.
3. 실패: 문구 · 404 면 목록 다시 받기 · 버튼 풀기.
4. 성공: 모달 닫기 → `selected` 를 새 값으로 → 그 줄을 `buildRow` 로 다시 만들어 바꿔 끼움.

5. 왜 이렇게

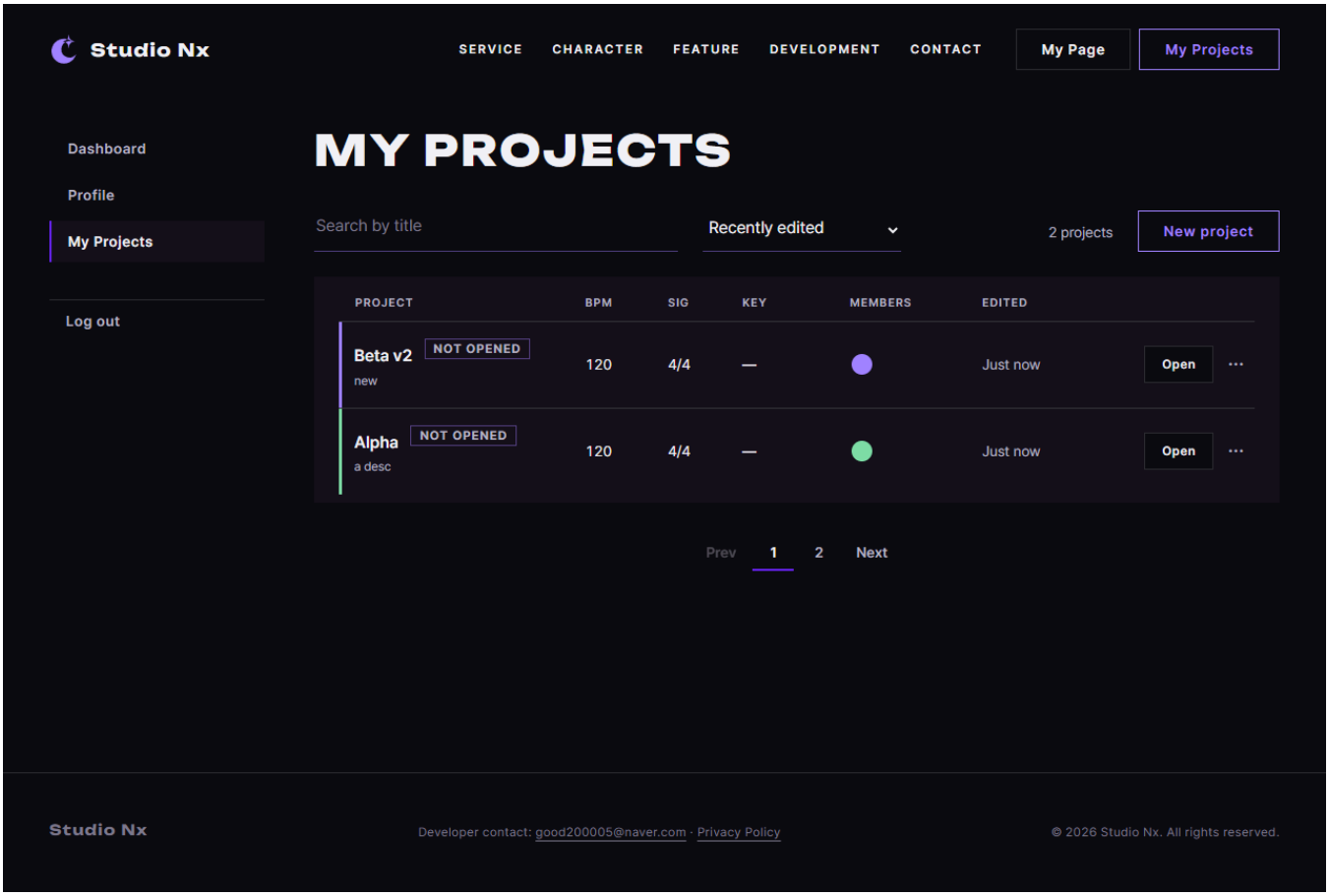
선택	이유
그 줄만 다시 그림	템플릿 · 목록 전체를 다시 받지 않아도 응답(`ProjectItem`)에 줄에 필요한 값이 모두 있음 (P1-13 62)
자리를 옮기지 않음	최근 수정순이면 원래 맨 위로 가야 하지만, 고친 줄이 눈앞에서 사라지면 사용자가 결과를 확인하기 어려움 → 다음 새로고침 · 검색 때 올라감
`buildRow` 를 다시 씀	`Not opened` · 역할 · 잠금 등 줄 규칙이 31 과 똑같이 적용됨 (`data-title` 도 새 값 → 다음 Edit 에 새 값이 채워짐, 시험 02)
공백 제목은 보내지 않음	39 와 같은 이유
404 면 목록 다시 받기	지워진 프로젝트 줄이 남아 있으면 또 누르게 됨
설명은 비우면 `null`	수정 칸을 비우는 것 = 설명 지우기 (P1-13 60 의 표와 같음)

6. 시험 (실제 myProject.html · 실제 서버 · 5개)

프로젝트 `Alpha`(설명 `a desc`) · `Beta` 를 만들고, … → Edit details 로 모달을 연 뒤 저장했다.

#	한 일	결과
01	Beta → `' Beta v2 '` · `new`	모달 닫힘 · 줄 `Beta v2 · new` (첫 줄 그대로) · `Alpha` 줄 그대로 · 서버 `Beta v2` · `new` · PATCH 1번
02	다시 Edit → 설명 공백	모달에 새 값(`Beta v2` · `new`)이 채워져 있음 · 저장 뒤 `No description` · 서버 `null`
03	Alpha 제목 공백	`Enter a title.` · PATCH 0번 · 모달 그대로
04	Edit 을 연 뒤 다른 곳에서 Alpha 를 지우고 저장	`Project not found.` · 목록을 다시 받아 Alpha 줄 사라짐
05	서버 500 (가짜)	문구 · 모달 그대로 · 버튼 풀림

캡처 — 저장 뒤 (`Beta v2 · new`)



JS 41 — `initDeleteProject()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js` · `studionx/frontend/myProject.html`
작업	P1-14 내 프로젝트 화면 — 프로젝트 삭제 · PRD 01 FR-125 · FIX-01 확인창 (2026-10-07 결정)
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	40 `initEditProjectForm()` · 42 `loadInitialList()`

1. 하는 일

`···` → **Delete** 를 누르면 확인창(`confirmDialog`, FIX-01 20)을 띄우고, **Delete project** 를 고르면 `DELETE /projects/{id}`(P1-13 61, 소프트 삭제) 후 목록을 다시 받는다.

상황	결과
Delete → 확인창	제목 `Delete project?` · 문장 `"P03" will be removed from your projects. Collaborators will lose access too.` · 빨간 `Delete project` 버튼 (처음 커서는 Cancel)
Cancel · Esc · 바깥 클릭	아무 일 없음
Delete project	삭제 → 같은 검색 · 정렬 · 페이지로 다시 받음 (`12 projects` → `11 projects`)
2쪽의 마지막 한 줄을 지움	빈 2쪽을 그리지 않고 **1쪽으로**
OWNER 가 아님	Delete 버튼이 잠겨 있어(31) 확인창도 뜨지 않음
서버 오류	알림창 `Couldn't delete` + 서버 문구 · 버튼 다시 켜짐
그사이 다른 곳에서 지워짐 (404)	알림창 `Project not found.` + 목록 다시 받음

HTML 도 바꿈 (2026-10-07 결정대로): `#delete-project-modal` `<dialog>` 를 지우고, 목록 줄 · 틀(`<template>`)의 Delete 버튼 7곳에서 `command="show-modal" commandfor="delete-project-modal"` 을 뺐다. **34 `initRowMenus` 에서도** 삭제 모달 제목을 채우던 세 줄을 지웠다 (모달이 없어져서).

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`myproject.js` 맨 아래, 이미 있음)	`initDeleteProject();` — 목록에 한 번

확인창(`confirmDialog`): 브라우저 기본 `confirm()` 대신 쓰는 SweetAlert2 창 (FIX-01 20). `await confirmDialog({...})` 는 확인이면 `true`, 취소 · Esc · 바깥 클릭이면 `false`. `danger: true` 면 확인 버튼이 빨강이고, 처음 커서가 **Cancel** 에 있어 Enter 를 바로 눌러도 지워지지 않는다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio, openBlankStudio } from '../co
re/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts, confirmDialog, showAlert } from '../core/dialog.js';
```

`confirmDialog` · `showAlert` 를 이번에 추가했다. 이 함수는 `api` · `confirmDialog` · `showAlert` 와 같은 파일의 `current` · `loadProjects` · `renderProjects` · `renderPager` · `refreshList` 를 쓴다.

함수

```
export function initDeleteProject() {
  const list = document.getElementById('project-list');
  if (!list) return;

  list.addEventListener('click', async function (event) {
    const button = event.target.closest('[data-action="delete"]');
    if (!button || button.disabled) return;
    const row = button.closest('li[data-project-id]');
    const id = row.dataset.projectId;
    const title = row.dataset.title;

    const ok = await confirmDialog({
      title: 'Delete project?',
      text: '“' + title + '” will be removed from your projects. Collaborators will lose access too.',
      confirmText: 'Delete project',
      danger: true,
    });
    if (!ok) return;

    button.disabled = true;   // 두 번 누름 방지 (성공하면 줄이 다시 그려짐)
    try {
      await api.delete('/projects/' + id);
    } catch (err) {
      button.disabled = false;
      await showAlert('Couldn’t delete', err.message, { tone: 'error' });
      if (err.status === 404) refreshList({ query: current.query, sort: current.sort, page: current.page
});
      return;
    }

    // 같은 조건으로 다시 받기 — 마지막 줄을 지워 그 페이지가 비면 (빈 화면을 그리지 않고) 한 페이지 앞으로
    const page = current.page;
    const data = await loadProjects({ query: current.query, sort: current.sort, page: page });
    if (!data) return;
    if (data.items.length === 0 && page > 1) {
      refreshList({ query: current.query, sort: current.sort, page: page - 1 });
      return;
    }
    renderProjects(data);
    renderPager(data.total, data.page, data.size);
```

```
  });
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const { projectId: id, title } = row.dataset;
text: `“${title}” will be removed from your projects. Collaborators will lose access too.`,
const { query, sort, page } = current;
```

최신 문법	실제 코드	뜻
`const { projectId: id, title } = row.dataset`	`const id = row.dataset.projectId` 등 두 줄	칸 꺼내며 이름 바꾸기
템플릿 문자열	`'“' + title + '” will …'`	글자 잇기
`const { query, sort, page } = current`	`current.query` 등 그대로	칸 꺼내기

문법 풀이

import 문법	뜻
`import { …, confirmDialog, showAlert } from '../core/dialog.js'`	확인창 (true/false) · 알림창 (FIX-01 20 · 18)

식	뜻	예
`event.target.closest('[data-action="delete"]')`	누른 곳에서 Delete 버튼 찾기 (목록 하나에 위임, 34 와 같음)	
`button.disabled`	잠긴 버튼 — 브라우저는 잠긴 버튼의 클릭을 보내지 않지만 한 번 더 확인	EDITOR
`const ok = await confirmDialog({…})`	사용자가 고를 때까지 기다림	true / false
`api.delete('/projects/' + id)`	DELETE 요청 — 성공이면 204 (본문 없음, `null`)	
`data.items.length === 0 && page > 1`	지운 뒤 그 쪽이 비었고 1쪽이 아님	2쪽의 마지막 한 줄

4. 흐름

1. Delete 버튼이 아니거나 잠겼으면 끝 → 그 줄의 번호 · 제목.
2. 확인창 → 취소면 끝.
3. 버튼 잠금 → `DELETE`.
4. 실패: 버튼 풀기 · 알림창 · 404 면 목록 다시 받기.
5. 성공: 같은 조건으로 받기 → 그 쪽이 비었고 1쪽이 아니면 앞 쪽으로, 아니면 그대로 그리기.

5. 왜 이렇게

선택	이유
HTML 모달 대신 `confirmDialog`	2026-10-07 결정 (FIX-01) — 모든 확인창을 같은 모양으로 · 휴대폰에서는 아래 시트
번호 · 제목을 **누른 줄**에서 바로 읽음	34 의 `selected` 는 다른 듣기가 먼저 불려야 채워짐 → 듣는 순서에 기대지 않음

선택	이유
처음 커서는 Cancel (`danger`)	되돌리기 어려운 일 — Enter 한 번에 지워지지 않게
실패는 알림창	모달이 없어져 문구 칸(`#delete-project-error`)도 없음
빈 쪽을 **그리지 않고** 앞 쪽으로	처음엔 `refreshList` 로 빈 2쪽을 그린 뒤 1쪽으로 갔는데, 그 사이 "No projects yet" 가 잠깐 보였다 → 받은 결과를 먼저 보고 결정하도록 고침 (시험 03 에서 빈 안내가 한 번도 안 보였음을 확인)
404 면 목록 다시 받기	지워진 줄이 남지 않게 (40 과 같음)
누른 뒤 버튼 잠금	확인 뒤 응답 전 두 번 눌림 방지 (템플릿)

6. 시험 (실제 myProject.html · 실제 서버 · 7개 + 34 다시)

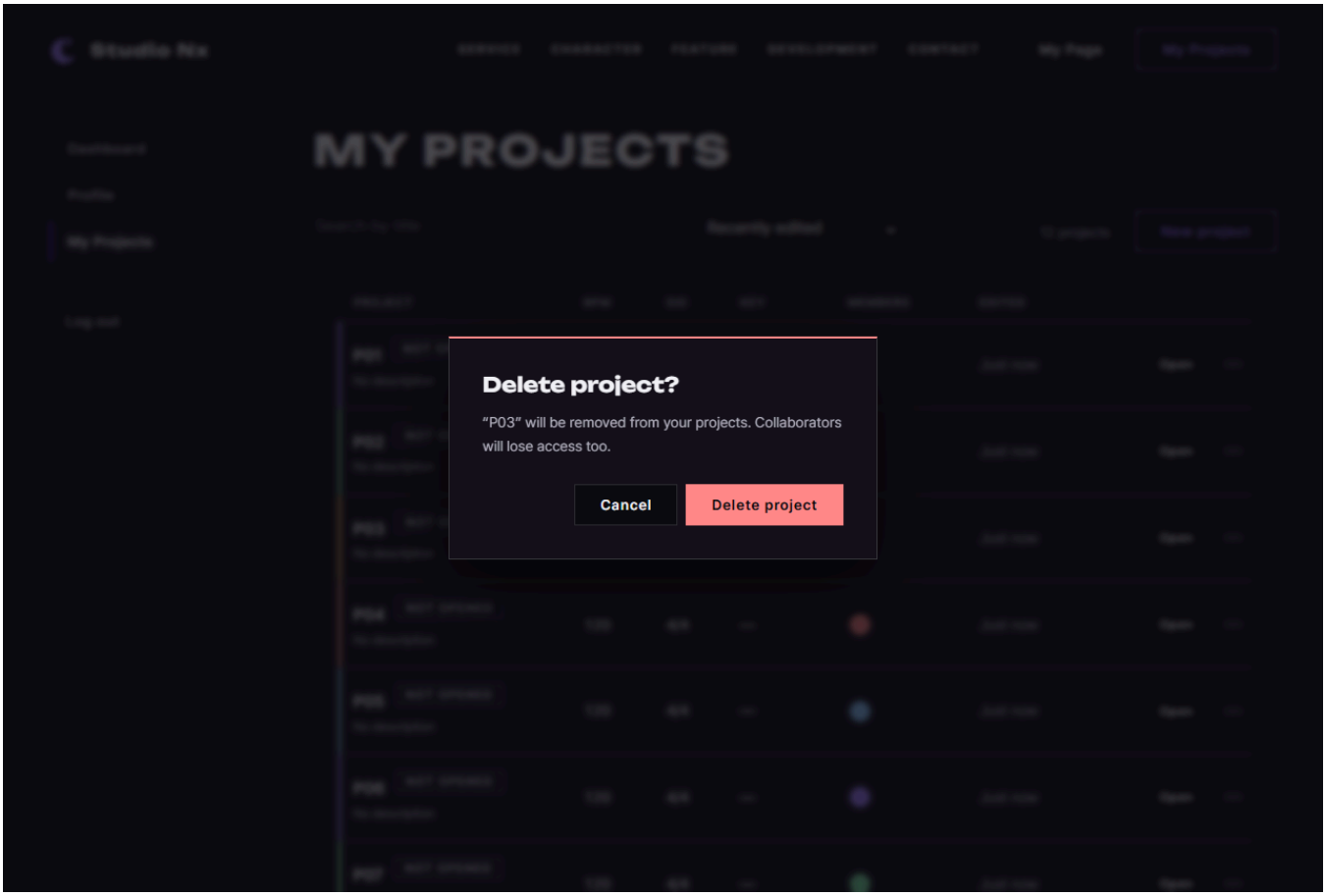
프로젝트 12개(`P01` ~ `P12`)를 이름순으로 그리고 `...` → Delete 를 눌렀다.

#	한 일	결과
01	P03 Delete	확인창 제목 · 문장 · 버튼 글자 맞음
01b	Cancel	DELETE 0번 · 12개 그대로 · 창 닫힘
02	다시 Delete → Delete project	DELETE 1번 · `11 projects` · P03 사라지고 P11 이 올라옴 · 서버 검색 0개
03	2쪽(P12 한 줄)에서 P12 삭제	1쪽 10개 · 주소 `?sort=title` · 페이지 줄 숨김 · **빈 안내가 한 번도 안 보임**
04	EDITOR 줄(가짜)의 Delete	확인창 안 뜸 · DELETE 0번
05	서버 500 (가짜)	알림창 `Couldn’t delete` · `Something went wrong. Please try again.` · 버튼 다시 켜짐 · 목록 그대로
06	확인창을 띄운 채 다른 곳에서 P02 를 지움 → Delete project	알림창 `Project not found.` → 목록 다시 받아 P02 사라짐

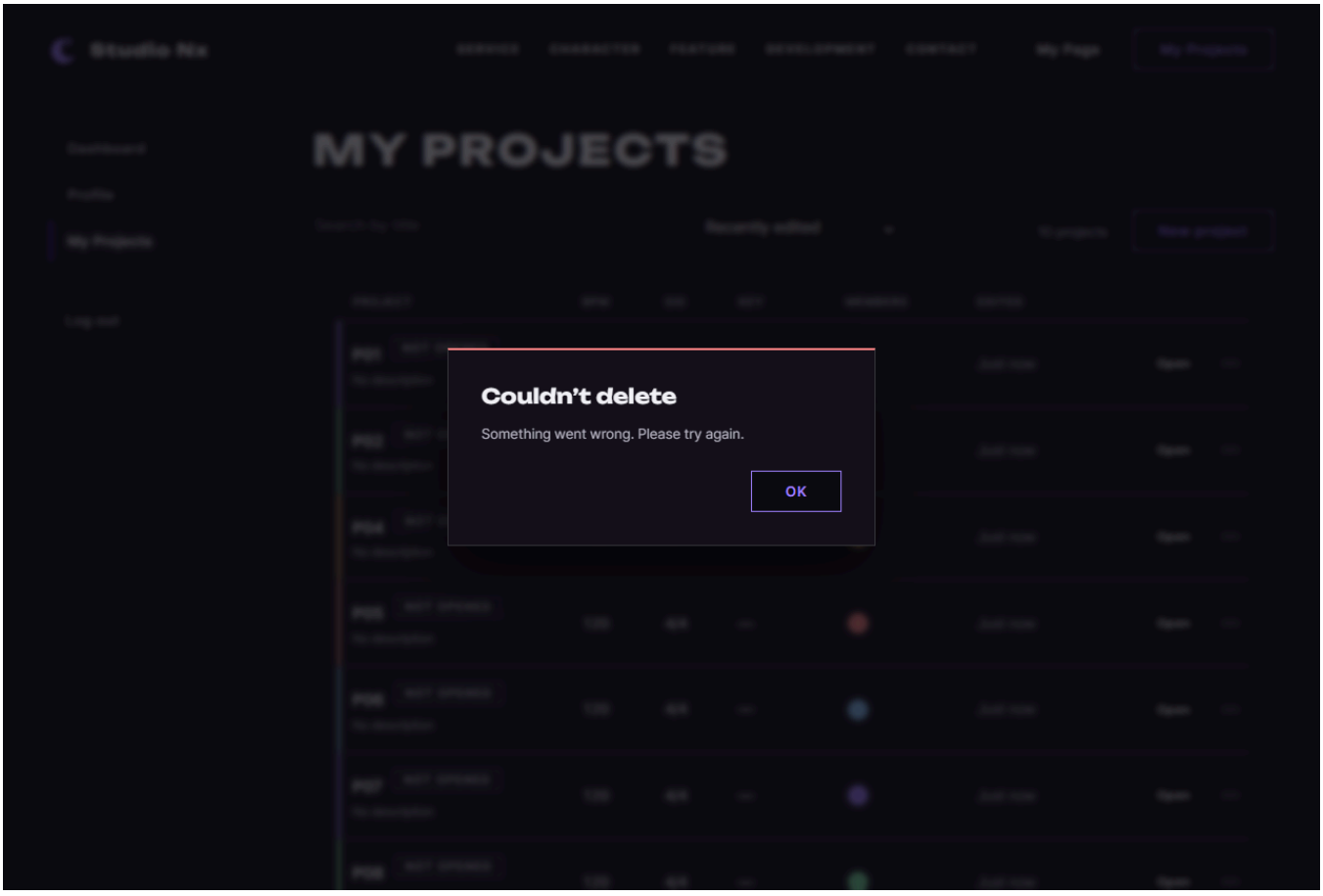
34 다시: 삭제 모달이 없어졌으므로 34 의 시험 03 을 "Delete → 확인창이 뜨고 `selected` 가 그 프로젝트" 로 바꿔 7개를 다시 돌렸다 — 모두 통과 (제목 `Neon "Rain" <x>` 도 확인창에 글자 그대로).

캡처

확인창:


MY PROJECTS
Delete project?
“P03” will be removed from your projects. Collaborators will lose access too.
Cancel
Delete project

서버 오류:


MY PROJECTS
Couldn’t delete
Something went wrong. Please try again.
OK

JS 42 — `loadInitialList()` · 페이지 시작

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면 — 페이지를 열 때 (로그인 확인 · 첫 목록 · 로그아웃 버튼) · PRD 01 FR-123
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	41 `initDeleteProject()` · (P1-14 끝 — 개요 51)

1. 하는 일

myProject.html 을 **열었을 때** 할 일을 마무리한다. 이 함수가 생기기 전에는 HTML 의 예시 줄 7개가 그대로 보였고, 로그인하지 않아도 페이지가 열렸다.

주소로 열기	결과
로그인 안 한 채 `myProject.html`	`home.html#login?next=%2FmyProject.html` — 로그인하면 돌아옴
`myProject.html`	최근 수정순 10개 · `12 projects` · `Prev 1* 2 Next` · 예시 줄 없음
`?query=p1&sort=title`	검색 칸 `p1` · 정렬 Name · `P10, P11, P12`
`?page=9` (2쪽까지)	**마지막 쪽(2)** 으로 · 주소 `?page=2`
`?sort=random&page=abc`	정렬 Recently edited · 1쪽 · 주소 깨끗 (30 이 고침)
Log out 버튼	로그아웃 → `home.html` (P1-10 27 `initLogoutButton`)

함께 바꾼 것

- 파일 맨 아래 시작 코드에 두 TODO 를 채움: `if (await requireLogin()) await loadInitialList();`
- 빈 `initLogoutButton` 템플릿 함수를 지우고 `core/auth-ui.js` 것을 import (P1-10 에서 세 화면 공용으로 만든 것). 쓰지 않게 된 `logout` import 도 뺌.

2. 언제 쓰나

호출하는 곳	화면
`myproject.js` 맨 아래 (페이지를 열 때 한 번)	로그인 확인 뒤 첫 목록

시작 순서 (파일 맨 아래)

```
await includePartials();          // 헤더 · 푸터 · 안내 모달 끼우기 (아래 init 들이 그 요소를 찾음)

initAuthState();
initModalFallback();
initFormAlerts();
initSearch();                     // 33
initNewProjectForm();             // 39
initRowMenus();                   // 34
initDesktopOnlyLinks(document.getElementById('project-list'));   // 38
initEditProjectForm();            // 40
initDeleteProject();              // 41
```

```
initLogoutButton();               // core/auth-ui.js (P1-10 27) — 세 화면 공용

// 로그인 확인 → 첫 목록 (로그인 안 했으면 home.html#login?next=… 로 옮겨 감)
if (await requireLogin()) await loadInitialList();
```

버튼 · 폼의 듣기를 먼저 걸고, 서버를 기다리는 일(로그인 확인 · 목록)은 **맨 끝**에 둔다 — 기다리는 동안에도 버튼이 반응한다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState, initLogoutButton } from '../core/auth-ui.js';
import { requireLogin, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio, openBlankStudio } from '../co
re/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts, confirmDialog, showAlert } from '../core/dialog.js';
```

`initLogoutButton` 을 추가하고 `logout` 을 뺐다. 이 함수는 같은 파일의 `refreshList`(33) · `current`(30) 를 쓰고, 시작 코드는 `requireLogin`(P1-02) · `initLogoutButton`(P1-10) 을 쓴다.

함수

```
export async function loadInitialList() {
  const list = document.getElementById('project-list');
  const count = document.getElementById('project-count');
  const pager = document.getElementById('project-pager');
  if (list) list.textContent = '';
  if (count) count.textContent = '';
  if (pager) pager.hidden = true;

  const params = new URLSearchParams(location.search);
  const options = {
    query: params.get('query') || '',
    sort: params.get('sort') || 'updated',
    page: params.get('page') || 1,
  };
  let data = await refreshList(options);
  if (data && data.items.length === 0 && data.total > 0) {
    options.page = Math.ceil(data.total / data.size);   // 없는 쪽 → 마지막 쪽
    data = await refreshList(options);
  }
  if (!data) return null;

  const query = document.getElementById('project-query');
  const sort = document.getElementById('project-sort');
  if (query) query.value = current.query;
  if (sort) sort.value = current.sort;
  return data;
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const params = new URLSearchParams(location.search);
const options = {
  query: params.get('query') ?? '',
  sort: params.get('sort') ?? 'updated',
  page: params.get('page') ?? 1,
};
document.getElementById('project-query')?.setAttribute('value', current.query);
```

최신 문법	실제 코드	뜻
`a ?? b`	`a || b`	없으면(null) 기본값 — 여기서는 빈 글자도 기본값으로 가므로 `||` 가 오히려 맞음
`el?.…`	`if (el) …`	있을 때만

문법 풀이

import 문법	뜻
`import { initAuthState, initLogoutButton } from '../core/auth-ui.js'`	로그아웃 버튼 (P1-10 27) — userInfo · myPage 와 같은 함수
`import { requireLogin } from '../core/auth.js'`	로그인이면 `true`, 아니면 로그인 화면으로 보내고 `false` (P1-02)

식	뜻	예
`new URLSearchParams(location.search)`	주소의 `?` 뒤를 칸별로	`?query=p1&sort=title`
`params.get('query') || ''`	없으면 빈 글자	`null` → `''`
`Math.ceil(data.total / data.size)`	마지막 쪽 번호	12 / 10 → 2
`if (await requireLogin()) await loadInitialList();`	로그인을 확인한 **뒤에만** 목록 요청	
맨 위의 `await` (모듈 맨 바깥)	`type="module"` 스크립트는 함수 밖에서도 `await` 를 쓸 수 있음	

4. 흐름

1. 예시 줄 · 개수를 지우고 페이지 줄을 숨김 (받는 동안 가짜 목록이 보이지 않게).
2. 주소에서 검색어 · 정렬 · 페이지를 읽어 목록 받기 (`refreshList`, 30 이 틀린 값을 고침).
3. 받은 쪽이 비었는데 전체는 있으면 → 마지막 쪽을 다시 받기.
4. 검색 칸 · 정렬 칸을 실제로 쓴 값(`current`)으로 맞춤.

5. 왜 이렇게

선택	이유
주소에서 읽음	30 이 주소에 남긴 값 — 새로고침 · 뒤로 가기 · 복사한 주소 · 페이지 링크 새 탭(32)에서 같은 목록
칸은 `current` 로 맞춤	주소의 틀린 값(`sort=random`)이 아니라 실제로 쓴 값을 보여 줌

선택	이유
없는 쪽 → 마지막 쪽	지운 뒤 북마크한 `?page=9` 같은 주소로 와도 빈 화면이 아니라 마지막 목록
받기 전에 예시 줄을 지움	HTML 예시(Summer Loop 등)가 실제 목록처럼 잠깐 보이지 않게
로그인 확인을 먼저	로그인 안 한 사람에게 목록 요청 → 401 → 로그인 이동이 두 번 일어나지 않게 (userInfo 와 같은 모양)
`initLogoutButton` 은 import	P1-10 에서 세 화면 공용으로 만들어 둔 것 — 템플릿 주석대로 이 파일의 빈 함수는 지움

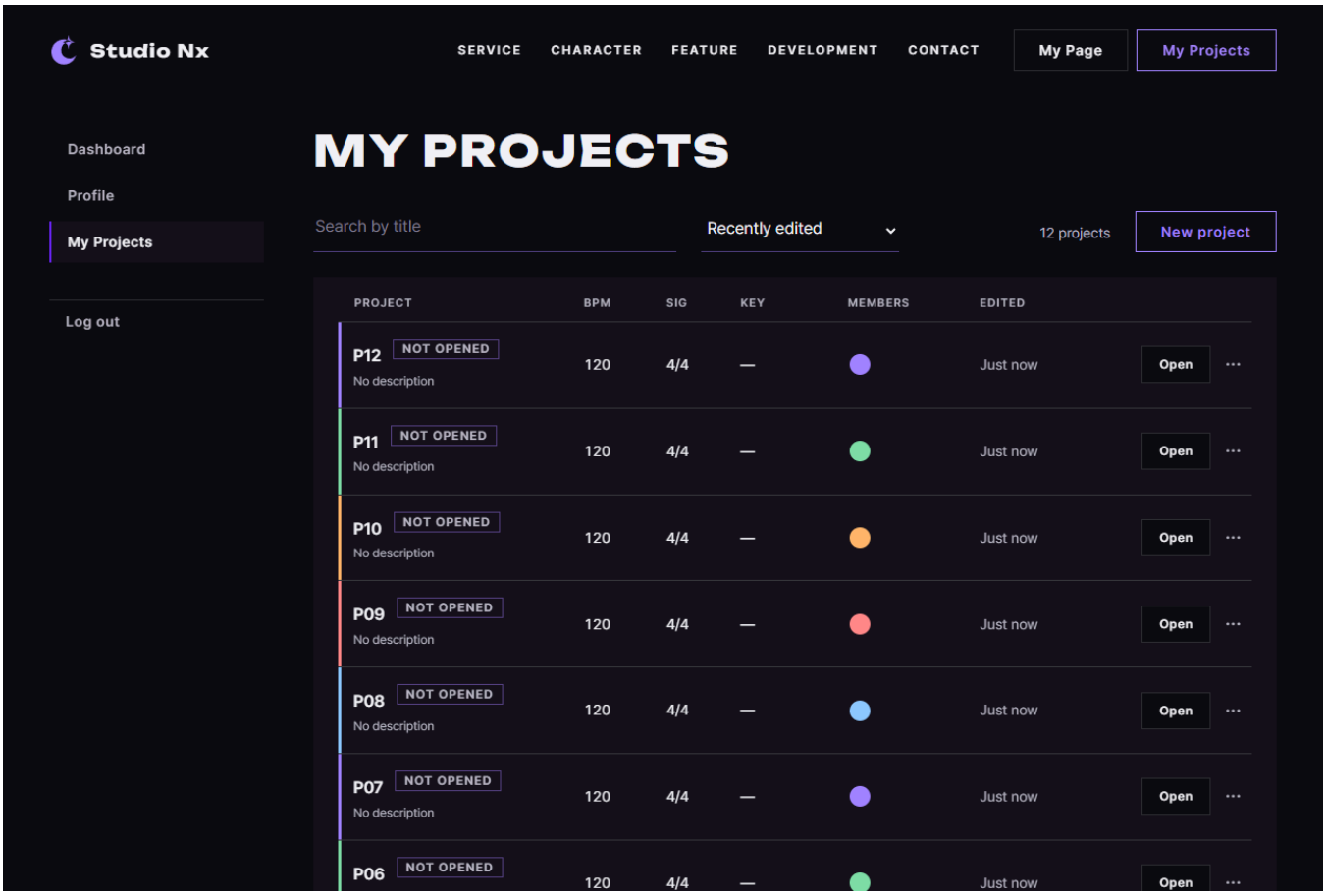
6. 시험 (실제 myProject.html 을 주소로 열기 · 실제 서버 · 7개 + 앞 시험 다시)

이번에는 함수를 직접 부르지 않고 **페이지를 주소로 열기만** 했다.

#	한 일	결과
01	로그인 안 한 채 열기	`home.html#login?next=%2FmyProject.html`
02	로그인 · 프로젝트 12개 → 열기	`P12 … P03` 10줄 · `12 projects` · `Prev 1* 2 Next` · 검색 빈 칸 · 정렬 updated · 예시 줄 없음
03	`?query=p1&sort=title`	검색 칸 `p1` · 정렬 title · `P10, P11, P12` · `3 projects` · 페이지 줄 숨김
04	`?page=9`	`P02, P01` · `Prev 1 2* Next` · 주소 `?page=2`
05	`?sort=random&page=abc`	정렬 updated · 1쪽 · 주소 `myProject.html`
06	Log out 클릭	`home.html`
07	로그아웃 뒤 다시 열기	로그인 화면으로

앞 시험 다시 (페이지가 이제 스스로 목록을 받으므로): 30(14) · 33(6) · 39(7) · 41(7) 을 다시 돌려 결과가 모두 그대로임을 확인했다.

캡처 — 로그인 뒤 열었을 때


Studio Nx
SERVICE
CHARACTER
FEATURE
DEVELOPMENT
CONTACT
My Page
My Projects
Dashboard
Profile
My Projects
Log out
MY PROJECTS
Search by title
Recently edited
12 projects
New project
PROJECT
BPM
SIG
KEY
MEMBERS
EDITED
P12 NOT OPENED
No description
120
4/4
—
Just now
Open
P11 NOT OPENED
No description
120
4/4
—
Just now
Open
P10 NOT OPENED
No description
120
4/4
—
Just now
Open
P09 NOT OPENED
No description
120
4/4
—
Just now
Open
P08 NOT OPENED
No description
120
4/4
—
Just now
Open
P07 NOT OPENED
No description
120
4/4
—
Just now
Open
P06 NOT OPENED
120
4/4
—
Just now
Open